GE Digital

Automation Management Platform

4.1.4

Installation Guide

Copyright

Copyright 2024, General Electric Company and/or its affiliates ("GE"). All Rights Reserved. This document is the confidential and proprietary information of GE and may not be reproduced, transmitted, stored, or copied in whole or in part, or used to furnish information to others, without the prior written permission of GE.

Contents

1. About This Document **6**
1.1. Purpose of This Document 6
1.2. Who Should Use This Document 6
1.3. Structure of This Document 6
1.4. For More Information 6
2. Deployment environment **8**
2.1. CI/CD Components 8
2.2. Connectivity type 8
2.3. Application type 8
2.4. Prerequisites for CI/CD setup 9
2.5. Setting up the Components 9
2.6. Recommended Practices 9
2.7. Conclusion 10
3. System Overview **11**
3.1. AMP Toolset 11
4. Installation Overview **13**
4.1. Installation Process 13
4.2. Post Installation 13
5. Quick Start Installation **14**
5.1. Pre-Requisites 14
5.1.1. Kubernetes 14
5.1.1.1. Ingress 14
5.1.1.2. Storage 14
5.1.2. AMP Images and Helm Chart 15
5.1.3. Installation Bootstrap/Execution Machine 15
5.1.3.1. Helm CLI 15
5.2. Prepare AMP Helm Chart Configuration 15
5.3. Deploy AMP 18
5.4. Validating the AMP deployment 19
5.5. What's Next 22
6. Installation **24**
6.1. Pre-Requisites 24
6.1.1. Kubernetes 24
6.1.2. AMP Images and Helm Chart 24
6.1.3. Ingress 24
6.1.4. Storage 24
6.1.5. Installation Bootstrap/Execution Machine 25
6.2. Prepare Helm Chart Configuration 25
6.3. Install Helm Chart 27
7. Uninstalling Chart **28**
8. Configuration **29**
8.1. Certificates manager 29
8.1.1. CA Issuer 29
8.1.1.1. 1. Create secrets using the Cert-Manager pem/crt and key. 29
8.1.1.2. 2. Create secret using the Customer root ca. 29

8.1.1.3. 3. Install AMP with the following changes to your *values.yaml*: 29
8.1.2. ACME Issuer 29
8.2. Trust Manager 30
8.2.1. Bundle 30
8.3. User Accounts 33
8.3.1. Gitea 33
8.3.2. Jenkins 34
8.3.3. Nexus 35
8.3.4. LDAP and SAML Authentication 35
8.4. Trusting Certificates in AMP Components 36
8.4.1. Using a secret to store certificates 36
8.4.1.1. Jenkins Controller 36
8.4.1.2. Jenkins Agent 38
8.4.1.3. Nexus Repository Server 40
8.4.2. Using a trust-manager bundle 42
8.4.2.1. Jenkins Controller 43
8.4.2.2. Jenkins Agent 44
8.4.2.3. Nexus Repository Server 45
8.4.2.4. Gitea 46
8.4.2.5. Argo CD 46
8.5. Using Secrets with Default/Admin Users 48
8.5.1. Jenkins Admin User credentials into a Kubernetes Secret 48
8.5.2. Gitea Admin User credentials into a Kubernetes Secret 48
8.5.3. ArgoCD Admin User credentials into a Kubernetes Secret 49
8.5.4. Nexus Admin User password into a Kubernetes Secret 50
9. Airgap Installation 52
9.1. Prerequisites 52
9.1.1. Bootstrap/Execution machine: 52
9.1.2. Target machine(s): 52
9.2. Instructions 52
9.2.1. Step #1: AMP Airgap installation preparation 53
9.2.2. Step #2: AMP installation 55
9.2.3. Step #3: AMP Airgap post-installation (optional) 55
10. Upgrade 56
10.1. Upgrade Documentation 56
10.2. Overview 56
10.3. Image Availability 56
10.4. Prepare AMP Installation Upgrade 56
10.5. Apply The AMP upgrade 57
11. Breaking Changes 58
11.1. AMP 4.1.1 (and argo_cd_v3.0.11) 58
11.2. AMP 4.1.0 (and *ge-gitea* 12.0.0) 58
11.3. AMP 4.0.0 (and *ge-nexus* 2.0.0): Sonatype Nexus Repository, OrientDB End Of Support 58
11.3.1. Migration Steps 58
11.4. AMP 3.1.2 (and *ge-jenkins* 5.4.2): Jenkins controller and PinK agent upgraded to use Java 17 61
11.5. AMP 3.0.0 (and *ge-gitea* 10.1.3): Gitea 10.x Helm Values 61

12. Security Considerations **62**
12.1. AMP 62
12.1.1. Open Ports 62
12.2. Installation Bootstrap\Execution machine 62
12.2.1. Open Ports 62
13. Secret Management **63**
13.1. Types of Credentials 63
13.2. Secret Masking 63
13.3. How to Add a New Secret 63
13.4. Using a Stored Secret in a Jenkins Job 63
14. Troubleshooting **65**
14.1. Certificates Are Not Trusted By Browser 65
14.2. Jenkins - max_user_namespaces Error 66
14.3. Jenkins - Timing Out While Pulling Docker Image 66
14.4. AMP - Checking the Chart Version from Current Deployment 66
15. Licenses **67**
15.1. Third Party Components 67
15.2. Third Party Component Premium Licenses 67
16. Migration from PDI to Helm AMP **68**
16.1. Overview 68
16.2. Image Availability 68
16.3. Prerequisites 68
16.4. Prepare AMP Installation Migration 68
16.5. Apply The AMP migration 71
16.5.1. Step #1 - Uninstalling PDI-based AMP components 71
16.5.2. Step #2 - Updating Cert. Manager CRDs 72
16.5.3. Step #3 - Changing PVCs attributes 72
16.5.4. Step #4 - Installing Helm-based AMP 72
16.6. Restore back-ups (where required) 72

1. About This Document

This document is supplied as a part of GE's Automation Management Platform (AMP) product.

1.1. Purpose of This Document

This document provides information about installing and maintaining the Automation Management Platform.

1.2. Who Should Use This Document

This document is intended for individuals who are responsible for installing and maintaining the Automation Management Platform on end users' infrastructure. It is also intended for system developers or operators who will manage or develop GE applications using AMP.

1.3. Structure of This Document

This document is structured in the following manner:

- About this Document - introduces this guide
- System Overview - describes AMP architecture and its purpose
- Installation - provides an overview of a typical AMP installation
- Quick Start - provides a short step-by-step AMP installation
- Air-Gapped Installation - describes how to install AMP on an environment which has no access to GE network
- Configuration - describes how to configure AMP
- Upgrade - describes how an AMP installation is upgraded
- Security Considerations - describes security considerations and open ports for AMP
- Secret Management - describes guidelines for storing secrets
- Troubleshooting - provides troubleshooting information
- Tables - includes tables of information, including a 3rd party component list.

1.4. For More Information

For more information about the Automation Management Platform (AMP) components, refer to the following:

- GE Nexus Chart Documentation
- GE Jenkins Chart Documentation
- GE Gitea Chart Documentation
- GE Cert Manager Chart Documentation

- GE ArgoCD Chart Documentation

2. Deployment environment

Continuous Integration and Continuous Deployment (CI/CD) embodies a set of practices that enable teams to automate the process of building, testing, and deploying software changes to production. Also, this enables delivering code, artifacts, implicitly new releases much faster. This document provides the guidelines for setting up the CI/CD infrastructure as a standard for GridOS applications.

Note these are general guidelines and can be adapted based on project needs.

2.1. CI/CD Components

A typical CI/CD GridOS environment would consist of:

1. Source Code Management (SCM) Tool: Used to track changes and manage source code.
2. Orchestrator: Used to define, automate, and manage the CI/CD pipelines. This tool integrates with the SCM Tool.
3. Repository Manager: Store and manage build artifacts, dependencies, and container images in a single place which is referenced by the code in SCM and accessed by the Orchestrator, if needed.
4. Certificate Manager: Manage the certificates used in the infrastructure for all applications.
5. Secrets manager (Optional): Additional/instead of the Kubernetes Secrets there is a possibility to integrate with a Secret Manager tool in order to add another layer of security.

2.2. Connectivity type

When categorizing by connectivity type, there are two main scenarios:

- Internet connectivity - when the environment can communicate over the internet, having access to GE sources like Grid Artifactory
- No internet connectivity (Airgapped) - all required artifacts need to be transferred asynchronously through the means agreed on the project

2.3. Application type

We consider two main types of applications are used in GE :

1. Monolith Applications: Refers to a traditional software architecture where all components of the application (e.g., business logic, database access, user interface) are tightly coupled and deployed as a single unit. Typically, monoliths are more challenging to scale and maintain compared to microservices due to their interconnected nature.
2. Microservices-based/Containerized Applications: Refers to an application architecture that leverages Kubernetes (or simply containers) for deployment and orchestration. Applications deployed on Kubernetes are often broken down into smaller, independent microservices, each running in its own container. Kubernetes provides the ability to manage these microservices efficiently, enabling features like scaling, rolling updates, and self-healing.

The long term plan is to migrate the Monolith applications towards Kubernetes Applications given the enumerated benefits. Depending on application type and its method of deployment, one tool of

choice can be more suitable than another.

2.4. Prerequisites for CI/CD setup

Deployment for GridOS application will rely on a set of prerequisites:

1. A Kubernetes cluster where the applications will be deployed.
2. A DNS server which will be used for accessing the applications
3. Access to the artifacts used for setup:
    1. When having internet connectivity, access to GE and/or external artifacts, when needed.
    2. For the airgapped scenario, a a prepacked bundle containing all required artifacts and a secure method to transfer them between GE environment and customer environment.
4. (Dependent on setup) Intermediate or Root Certificate Authority certificate, when issued by the customer

2.5. Setting up the Components

A high-level overview of setting up the components for both scenarios would involve:

- Check the infrastructure is meeting the prerequisites for deployment
- Configure the Secrets Manager (if any)
- Install the CI/CD tools on the Kubernetes cluster.
- Ensure the UIs are accessible and authentication is successful.
- Check if certificates are applied correctly.
- Set up access controls and permissions to manage user access.
- Create/upload the repositories that host the source code of the applications, if not provided as an option already by the SCM tool.
- Load the required artifacts onto the repository manager.
- Configure the CI/CD tool to fetch code from the SCM and define the pipelines.

2.6. Recommended Practices

- Apply the **principle of least privilege** when granting access to applications and to infrastructure
- Store **code only** on **SCM tool**
- Artifacts should be stored only in Repository Manager
- Preflight builds should be enabled, no change should be merged to main branch unless prior tested & passed
- The tools of choice should be on updated versions, CVE scanned/mitigated
- Credentials should be kept securely from unauthorized use
- Backups should be taken regularly (snapshots/+data backup)

- Scale when needed, depending on load
- Ensure HA when dealing with critical apps
- CI/CD deployment sources should be kept securely - in order to allow redeployment

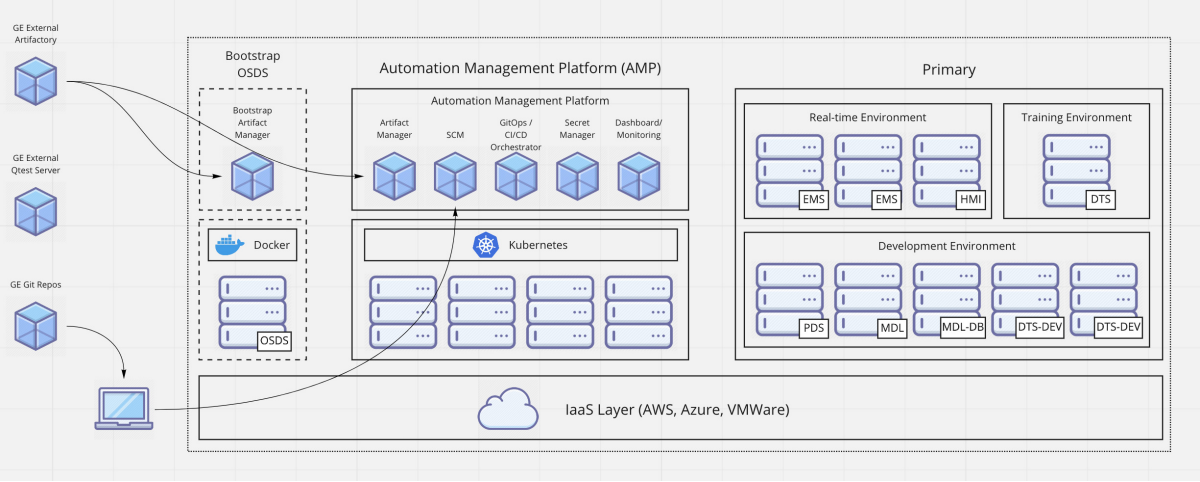
2.7. Conclusion

Deploying a robust CI/CD infrastructure is crucial for streamlining the development and deployment processes. By following the guidelines outlined in this documentation, you can set up a CI/CD process, whether you there is internet connectivity or operating in an airgapped environment.

3. System Overview

The Automation Management Platform (AMP) is a Development, CI and Deployment Environment in a Box. It contains everything needed to enable Continuous Delivery from GE to a remote premises. AMP provides tools to support 'GitOps', where:

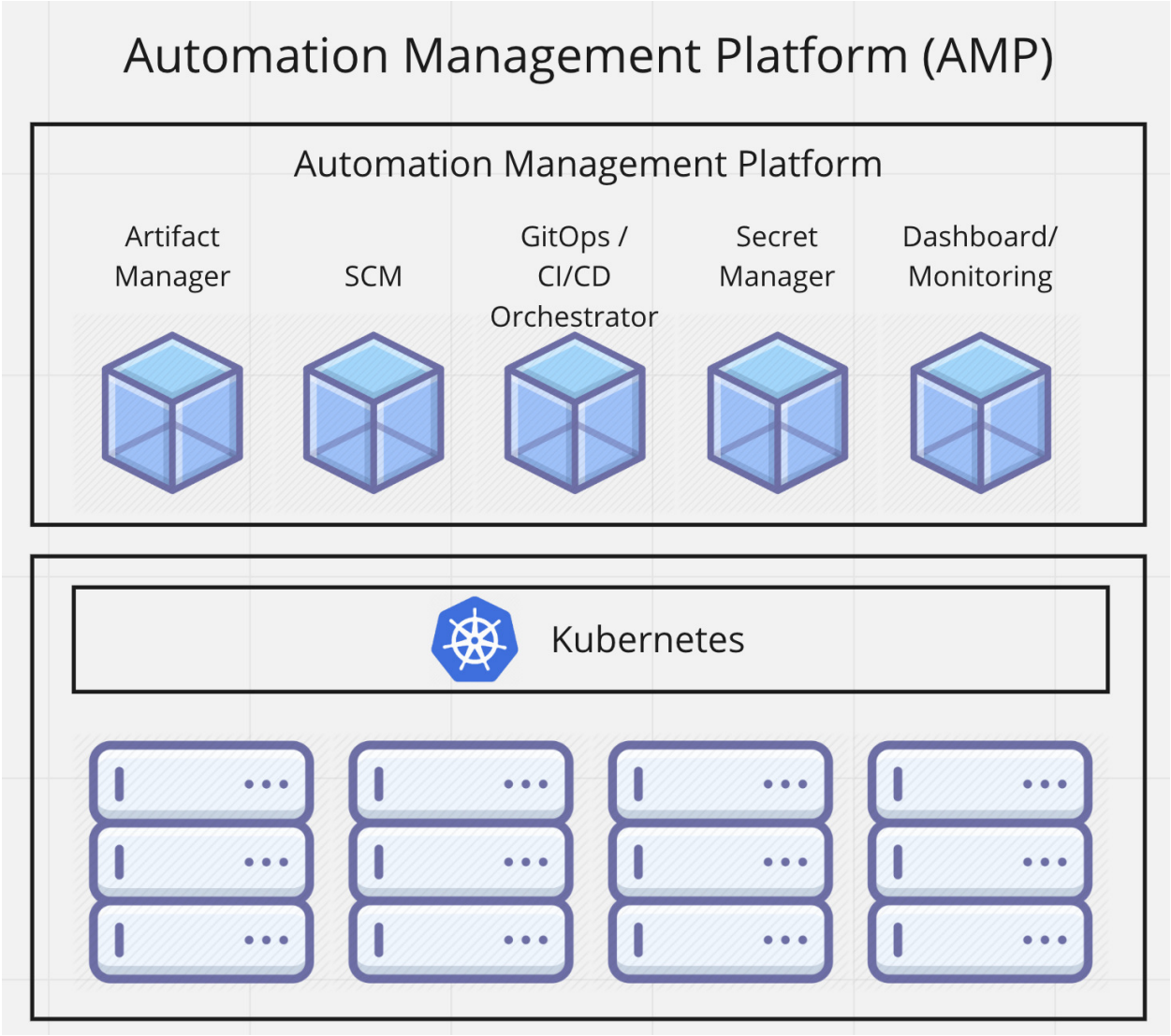
- A team can work collaboratively on a description of the Desired State of the infrastructure and system
- The description of the Desired State is stored in a source control system, ensuring audit history and promoting peer review of changes
- A change to the Desired State will automatically run pipeline processes to ensure the state of the system is reconciled to match the Desired State



AMP runs in a Kubernetes cluster, and can control application deployments across the infrastructure - be it physical machines, virtual machines or kubernetes-managed containers. It can also work with Infrastructure-as-a-Service platforms to create and manage the infrastructure itself. At the heart of AMP is a source control system for holding Desired State, an artifact repository for holding installers needed for the Desired State and an orchestrator for making that Desired State real.

3.1. AMP Toolset

AMP contains preconfigured tools which support GE's development and deployment best practice.



The current toolset comprises:

Purpose	Tool
Artifact Storage	Nexus
Docker Image Registry	Nexus
Build Orchestration	Jenkins
Source Control	Gitea
Certificate manager	Cert-manager

Future versions of AMP will manage secrets and dashboarding. Currently, Jenkins provides the basic requirements for secret management and dashboarding.

4. Installation Overview

AMP uses Helm to deploy its components to an existing Kubernetes Cluster.

4.1. Installation Process

The installation requires the following steps:

- Ensure Pre-requisites
  - Kubernetes
  - Images
  - Ingress
  - Storage
  - Installation Bootstrap/Execution Machine
- Prepare Helm Chart Configuration
- Install Helm Chart

4.2. Post Installation

After running the installation, the following steps should be completed:

- Validate (Test) AMP
- Configure AMP

5. Quick Start Installation

5.1. Pre-Requisites

5.1.1. Kubernetes

AMP is a set of containerised applications intended to run in Kubernetes. Before installing AMP you must have a suitable cluster to deploy to.

If you do not currently have a Kubernetes cluster consider whether you can use provided cluster in the Cloud, or whether you need full control/ownership of the infrastructure on which it resides. AWS and Azure (Microsoft) can quickly provision clusters for you. For creating clusters on your own infrastructure, GE maintains an installer for installing our preferred distribution of Kubernetes - RKE2.

5.1.1.1. Ingress

Incoming traffic to AMP will all be routed through a https 'Ingress'. To control this your cluster must have an Ingress Controller. Depending on how your Kubernetes cluster was supplied it is likely to already have an Ingress Controller. There is no Kubernetes type for Ingress Controllers, but by convention the running services and pods for an Ingress Controller would have 'ingress' in their name, so you can check for a running Ingress Controller with:

```
kubectl get service --all-namespaces | grep ingress
```

If no Ingress Controller is present then:

- either use the Foundation base install to get it on the system
- or make your own choice of ingress controller. A list of them are here.

5.1.1.2. Storage

AMP includes databases, and these will need persistent storage. A Kubernetes cluster can provide a number of Storage Classes to cater for different storage needs. Check what storage classes are available in your cluster with:

```
kubectl get sc
```

Note that AMP components are by default deployed as StatefulSets, which means that storage does not need to be shared between pods. This in turn means that ANY writable storage class will suffice. The Access Mode required is ReadWriteOnce, which is an available mode for all volume plugins.

Provisioner	URL	Notes
Local	https://github.com/rancher/local-path-provisioner	Already installed on RKE2 and Foundation. Easy setup, high performance, but cannot share storage between pods running on different Kubernetes nodes.

Provisioner	URL	Notes
In-Cluster NFS Server	https://kubernetes-sigs.github.io/nfs-ganesha-server-and-external-provisioner	If you prefer the ability to share storage across nodes and the convenience of not needing to install your own NFS server, this uses the local provisioner above to create and manage an NFS server inside the cluster.
External NFS Server	https://github.com/kubernetes-sigs/nfs-subdir-external-provisioner	If you prefer to have a centralized NFS server outside of the cluster, then this provisioner can be used.

5.1.2. AMP Images and Helm Chart

The containerised applications AMP runs are supplied as images along with an umbrella Helm chart which in turn installs the images with a series of sub-charts embedded inside the umbrella chart. These images can be pulled directly from the internet (Grid Artifactory - https://dig-grid-artifactory.apps.ge.com/). Customer environments are likely to be disconnected from the internet to some degree (air-gapped).

See the Air Gapped Installation documentation for details of how to supply images in these circumstances.

5.1.3. Installation Bootstrap/Execution Machine

AMP is the ideal location from which to manage deployments. However, a location is needed from which to deploy AMP. This is the Installation Bootstrap/Execution Machine, which has the following requirements:

- Should be a secure machine with limited user access
- Needs a Container Runtime (either Docker OR Podman)

Instructions for installing the required tools can be found at the following locations:

Tool	URL
Podman	https://podman.io/getting-started/
Docker	https://www.docker.com/get-started
Helm	https://helm.sh/docs/intro/install/

5.1.3.1. Helm CLI

Installation guide: https://helm.sh/docs/intro/install/

5.2. Prepare AMP Helm Chart Configuration

The AMP Helm Chart Configuration allows you to adapt parameters for the AMP deployment to your needs. This YAML file contains the values to be applied during an installation or upgrade using the Helm CLI. For more information, please refer to https://helm.sh/docs/helm/helm_install/ or

https://helm.sh/docs/helm/helm_upgrade/.

Step 1

Save the following content into a file named `solution-value-overrides.yaml`.

```
global:
## Public host name of the ArgoCD instance
  argoCDHostName: &argoCDHostName "argocd.amp.myco.com"
## Public host name of the Jenkins instance
  jenkinsHostName: "jenkins.amp.myco.com"
## Public host name of the Gitea instance
  giteaHostName: "gitea.amp.myco.com"
## Public host name of the Gitea instance
  nexusHost: "nexus.amp.myco.com"
  nexusUiHost: "ui.nexus.myco.com"
  nexusDockerPushHost: "nexus.push.amp.myco.com"
  nexusDockerPullHost: "nexus.pull.amp.myco.com"
  ingress:
    className: &ingressClassName "nginx"
  amp:
    storageClass: &ampStorageClass "local-path"
  certificate:
    issuer:
      name: &ampCertIssuer "amp-issuer"
  trust-manager:
    enabled: false

amp:
  jenkins:
    enabled: true
  gitea:
    enabled: true
  nexus:
    enabled: true
  certmanager:
    enabled: true
  argoCD:
    enabled: false

#####   Components Values   #####

ge-cert-manager:
  cert-manager:
    installCRDs: true
  trust-manager:
    crds:
      enabled: true
    secretTargets:
      enabled: true
      #It will enable writing trust bundles to Kubernetes Secrets as a target.
      authorizedSecretsAll: true
      #A list of secret names which trust-manager will be permitted to read and write across all
namespaces.
```

```
      authorizedSecrets: []
    app:
      trust:
        #The namespace used as the trust source. Note that the namespace must exist before installing
trust-manager.
        namespace: amp
      webhook:
        tls:
          approverPolicy:
            #To enable the approver-policy CertificateRequestPolicy allowing auto-approval of the
trust-manager webhook certificate.
            enabled: false
            #The namespace in which cert-manager was installed. Only used if
`app.webhook.tls.approverPolicy.enabled` is true..
            certManagerNamespace: amp

ge-argo-cd:
  argo-cd:
    server:
      ingress:
        hostname: *argoCDHostName
        ingressClassName: *ingressClassName
        annotations:
          cert-manager.io/issuer: *ampCertIssuer # amp-issuer

ge-jenkins:
  jenkins:
    persistence:
      storageClassName: *ampStorageClass
    controller:
      admin:
#        username: admin
        password: admin123
      ingress:
#        enabled: true
        ingressClassName: *ingressClassName
        annotations:
          cert-manager.io/issuer: *ampCertIssuer # amp-issuer

ge-gitea:
  persistence:
    storageClass: *ampStorageClass
  gitea:
    gitea:
      admin:
        username: 'root'
        password: "admin123"
    ingress:
      className: *ingressClassName
      annotations:
        cert-manager.io/issuer: *ampCertIssuer # amp-issuer

ge-nexus:
```

```
  persistence:
    storageClass: "{{ .Values.global.amp.storageClass }}"
  ingress:
    className: "{{ .Values.global.ingress.className }}"
    annotations:
      cert-manager.io/issuer: *ampCertIssuer # amp-issuer
```

Step 2

Override values from `solution-value-overrides.yaml` according to your solution requirements.

5.3. Deploy AMP

AMP is deployed on a kubernetes (k8s) cluster using the Helm CLI.

Step 1

Retrieve your target kubernetes (k8s) cluster access file: the *kubeconfig* files. This file can be inputted to the Helm CLI commands using the `--kubeconfig` flag followed by the path to the file.

Step 2

Install/Upgrade AMP onto the k8s cluster. The installation/upgrade of AMP presents two options

Using helm repository from GE Grid Artifactory

```
helm repo add AMP-helm https://dig-grid-artifactory.apps.ge.com/artifactory/virtual-helm
helm upgrade amp AMP-helm/AMP --install -n amp --create-namespace -f solution-value-overrides.yaml
```

- This option assumes internet access to GE Grid Artifactory and to the *virtual-helm* repository. Adapt the Helm chart repository URL accordingly.
- The target cluster must be bound to a image registry named *dig-grid-artifactory.apps.ge.com/virtual-docker* allow it to fetch the docker images required by the AMP deployment.
- This command will use the latest version of the AMP Helm chart. If looking to use a specific version, add to the command the `--version` flag followed by the version number.

Using the airgap AMP bundle

This approach assumes that a airgap AMP bundle has been copied onto a bootstrap machine (in the same network as the targeted cluster node) and unarchived. From the directory where the bundle got unpacked:

1. Prepare the airgap cluster

```
./airgap-pre-installation.sh --ip <target-ip> --key /path/to/ssh.pem --user <target-user>
```

- It requires a SSH connection to the target node based on an identity key (in PEM format).
- If no cluster existing on the target node and the all-in-one airgap AMP bundle is being used, add `--rke2` to the command to install an RKE2 cluster.

2. Install AMP

```
helm upgrade amp AMP-*.tgz --install -n amp --create-namespace -f value-overrides.yaml -f solution-value-overrides.yaml
```

> Because the `helm upgrade` command allows the use of multiple `--value`/`-f` flags, we take advantage of it by providing both the bundle-provided configuration values file and the one prepared earlier on.

5.4. Validating the AMP deployment

AMP deployment can be validated as follow:

Review the K8s resources created

View all resources that exist in the AMP namespace

```
kubectl get all -n amp
```

An example output is as follows.

```
NAME                                                      READY   STATUS      RESTARTS   AGE
pod/amp-argocd-application-controller-0                   1/1     Running     0          34m
pod/amp-argocd-applicationset-controller-9759fbfbb-c9sgw  1/1     Running     0          34m
pod/amp-argocd-dex-server-757fd95877-6rsbv                1/1     Running     0          34m
pod/amp-argocd-notifications-controller-74b58c7f64-v6vg4  1/1     Running     0          34m
pod/amp-argocd-redis-647fbd4f89-vk4st                     2/2     Running     0          34m
pod/amp-argocd-repo-server-8458c875f6-mtjcl               1/1     Running     0          34m
pod/amp-argocd-server-59f5546f8-g29kk                     1/1     Running     0          34m
pod/amp-cert-manager-7586bf9bc-b5sjc                      1/1     Running     0          34m
pod/amp-cert-manager-cainjector-568bb7b64c-bm9kf          1/1     Running     0          34m
pod/amp-cert-manager-startupapicheck-kstw6                0/1     Completed   0          32m
pod/amp-cert-manager-webhook-d8b85b86b-pn4rc              1/1     Running     0          34m
pod/amp-cluster-0                                         1/1     Running     0          34m
pod/amp-ge-nexus-0                                        1/1     Running     0          34m
pod/amp-ge-nexus-nexus-init-pwd-job-sv4xf                 0/1     Completed   0          32m
pod/amp-gitea-75c785f9fb-25pj9                            1/1     Running     0          34m
pod/amp-jenkins-0                                         2/2     Running     0          34m
pod/amp-postgres-operator-54d4864-6hrjr                   1/1     Running     0          34m
pod/amp-repo-config-job-4k785                             0/14    Error       0          17m
pod/amp-repo-config-job-5dddm                             0/14    Error       0          22m
pod/amp-repo-config-job-cv2c4                             0/14    Completed   0          29m
pod/amp-repo-config-job-kg7xv                             0/14    Error       0          27m
pod/amp-repo-config-job-n5f8n                             0/14    Error       0          28m
pod/amp-repo-config-job-sfdnl                             0/14    Error       0          28m
pod/amp-repo-config-job-smrph                             0/14    Error       0          25m

NAME                                          TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)
AGE
service/amp-argocd-applicationset-controller  ClusterIP   10.43.225.125   <none>        7000/TCP
```

```
34m
service/amp-argocd-dex-server                  ClusterIP   10.43.105.160   <none>
5556/TCP,5557/TCP            34m
service/amp-argocd-redis                       ClusterIP   10.43.225.253   <none>        6379/TCP
34m
service/amp-argocd-repo-server                 ClusterIP   10.43.7.112     <none>        8081/TCP
34m
service/amp-argocd-server                      ClusterIP   10.43.249.7     <none>
80/TCP,443/TCP               34m
service/amp-cert-manager                       ClusterIP   10.43.41.105    <none>        9402/TCP
34m
service/amp-cert-manager-cainjector            ClusterIP   10.43.169.170   <none>        9402/TCP
34m
service/amp-cert-manager-webhook               ClusterIP   10.43.159.245   <none>
443/TCP,9402/TCP             34m
service/amp-cluster                            ClusterIP   10.43.218.49    <none>        5432/TCP
34m
service/amp-cluster-config                     ClusterIP   None            <none>        <none>
34m
service/amp-cluster-repl                       ClusterIP   10.43.94.222    <none>        5432/TCP
34m
service/amp-ge-nexus                           ClusterIP   10.43.73.153    <none>
8081/TCP,8082/TCP            34m
service/amp-gitea-http                         ClusterIP   None            <none>        3000/TCP
34m
service/amp-gitea-ssh                          ClusterIP   None            <none>        22/TCP
34m
service/amp-jenkins                            ClusterIP   10.43.200.24    <none>        8080/TCP
34m
service/amp-jenkins-agent                      ClusterIP   10.43.35.189    <none>        50000/TCP
34m
service/amp-postgres-operator                  ClusterIP   10.43.84.181    <none>        8080/TCP
34m
service/ge-nexus-service                       ClusterIP   10.43.83.174    <none>
8081/TCP,8082/TCP,8083/TCP   34m

NAME                                                  READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/amp-argocd-applicationset-controller   1/1     1            1           34m
deployment.apps/amp-argocd-dex-server                  1/1     1            1           34m
deployment.apps/amp-argocd-notifications-controller    1/1     1            1           34m
deployment.apps/amp-argocd-redis                       1/1     1            1           34m
deployment.apps/amp-argocd-repo-server                 1/1     1            1           34m
deployment.apps/amp-argocd-server                      1/1     1            1           34m
deployment.apps/amp-cert-manager                       1/1     1            1           34m
deployment.apps/amp-cert-manager-cainjector            1/1     1            1           34m
deployment.apps/amp-cert-manager-webhook               1/1     1            1           34m
deployment.apps/amp-gitea                              1/1     1            1           34m
deployment.apps/amp-postgres-operator                  1/1     1            1           34m

NAME                                                     DESIRED   CURRENT   READY   AGE
replicaset.apps/amp-argocd-applicationset-controller-9759fbfbb   1         1         1       34m
replicaset.apps/amp-argocd-dex-server-757fd95877                 1         1         1       34m
replicaset.apps/amp-argocd-notifications-controller-74b58c7f64   1         1         1       34m
```

```
replicaset.apps/amp-argocd-redis-647fbd4f89                     1         1         1       34m
replicaset.apps/amp-argocd-repo-server-8458c875f6               1         1         1       34m
replicaset.apps/amp-argocd-server-59f5546f8                     1         1         1       34m
replicaset.apps/amp-cert-manager-7586bf9bc                      1         1         1       34m
replicaset.apps/amp-cert-manager-cainjector-568bb7b64c          1         1         1       34m
replicaset.apps/amp-cert-manager-webhook-d8b85b86b              1         1         1       34m
replicaset.apps/amp-gitea-75c785f9fb                            1         1         1       34m
replicaset.apps/amp-postgres-operator-54d4864                   1         1         1       34m

NAME                                                READY   AGE
statefulset.apps/amp-argocd-application-controller   1/1     34m
statefulset.apps/amp-cluster                         1/1     34m
statefulset.apps/amp-ge-nexus                        1/1     34m
statefulset.apps/amp-jenkins                         1/1     34m

NAME                                          STATUS     COMPLETIONS   DURATION   AGE
job.batch/amp-cert-manager-startupapicheck    Complete   1/1           5s         32m
job.batch/amp-ge-nexus-nexus-init-pwd-job     Complete   1/1           3m4s       32m
job.batch/amp-repo-config-job                 Failed     0/1           29m        29m

NAME                                                          IMAGE
CLUSTER-LABEL   SERVICE-ACCOUNT   MIN-INSTANCES   AGE
operatorconfiguration.acid.zalan.do/amp-postgres-operator   dig-grid-artifactory.apps.ge.com/virtual-
docker/spilo-cdp:15-3.0-p1-build.14   cluster-name    postgres-pod      -1              34m

NAME                                   TEAM   VERSION   PODS   VOLUME   CPU-REQUEST   MEMORY-REQUEST
AGE   STATUS
postgresql.acid.zalan.do/amp-cluster   amp    14        1      1Gi
34m   Running
```

Test the Helm installation

View all resources that exist in the AMP namespace

```
helm test amp -n amp
```

An example output is as follows.

```
NAME: amp
LAST DEPLOYED: Tue Feb  4 18:45:11 2025
NAMESPACE: amp
STATUS: deployed
REVISION: 1
TEST SUITE:     gitea-test-configmap
Last Started:   Tue Feb  4 18:53:00 2025
Last Completed: Tue Feb  4 18:53:00 2025
Phase:          Succeeded
TEST SUITE:     amp-jenkins-tests
Last Started:   Tue Feb  4 18:53:04 2025
Last Completed: Tue Feb  4 18:53:05 2025
Phase:          Succeeded
TEST SUITE:     jenkins-test-configmap
Last Started:   Tue Feb  4 18:53:00 2025
Last Completed: Tue Feb  4 18:53:00 2025
Phase:          Succeeded
TEST SUITE:     ge-nexus-test-configmap
Last Started:   Tue Feb  4 18:53:00 2025
Last Completed: Tue Feb  4 18:53:00 2025
Phase:          Succeeded
TEST SUITE:     amp-gitea-test-connection
Last Started:   Tue Feb  4 18:53:00 2025
Last Completed: Tue Feb  4 18:53:04 2025
Phase:          Succeeded
TEST SUITE:     amp-ui-test-ze5fj
Last Started:   Tue Feb  4 18:53:05 2025
Last Completed: Tue Feb  4 18:53:09 2025
Phase:          Succeeded
TEST SUITE:     ge-gitea-functional-tests
Last Started:   Tue Feb  4 18:53:09 2025
Last Completed: Tue Feb  4 18:53:16 2025
Phase:          Succeeded
TEST SUITE:     ge-jenkins-functional-tests
Last Started:   Tue Feb  4 18:53:16 2025
Last Completed: Tue Feb  4 18:54:23 2025
Phase:          Succeeded
TEST SUITE:     ge-nexus-functional-tests
Last Started:   Tue Feb  4 18:54:23 2025
Last Completed: Tue Feb  4 18:54:28 2025
Phase:          Succeeded
```

5.5. What's Next

Once the AMP deployment finalized:

- Review the needs for mapping the AMP component hostnames into the network DNS
- Refer to the 3rd-party official online documentation for component-specific configurations from their user interfaces
    - Cert-manager: https://cert-manager.io/docs/ & https://cert-manager.io/docs/trust/trust-

manager/

- Jenkins: https://www.jenkins.io/doc/
- Gitea: https://docs.gitea.com/
- Nexus: https://help.sonatype.com/en/sonatype-nexus-repository.html
- Argo CD: https://argo-cd.readthedocs.io/en/stable/

6. Installation

6.1. Pre-Requisites

6.1.1. Kubernetes

AMP is a set of containerised applications intended to run in Kubernetes. Before installing AMP you must have a suitable cluster to deploy to.

If you do not currently have a Kubernetes cluster consider whether you can use provided cluster in the Cloud, or whether you need full control/ownership of the infrastructure on which it resides. AWS and Azure (Microsoft) can quickly provision clusters for you. For creating clusters on your own infrastructure, GE maintains an installer for installing our preferred distribution of Kubernetes - RKE2.

6.1.2. AMP Images and Helm Chart

The containerised applications AMP runs are supplied as images along with an umbrella Helm chart which in turn installs the images with a series of sub-charts embedded inside the umbrella chart. These images can be pulled directly from the internet (Grid Artifactory - https://dig-grid-artifactory.apps.ge.com/). Customer environments are likely to be disconnected from the internet to some degree (air-gapped).

See the Air Gapped Installation documentation for details of how to supply images in these circumstances.

6.1.3. Ingress

Incoming traffic to AMP will all be routed through a https 'Ingress'. To control this your cluster must have an Ingress Controller. Depending on how your Kubernetes cluster was supplied it is likely to already have an Ingress Controller. There is no Kubernetes type for Ingress Controllers, but by convention the running services and pods for an Ingress Controller would have 'ingress' in their name, so you can check for a running Ingress Controller with:

```
kubectl get service --all-namespaces | grep ingress
```

If no Ingress Controller is present then:

- either use the Foundation base install to get it on the system
- or make your own choice of ingress controller. A list of them are here.

6.1.4. Storage

AMP includes databases, and these will need persistent storage. A Kubernetes cluster can provide a number of Storage Classes to cater for different storage needs. Check what storage classes are available in your cluster with:

```
kubectl get sc
```

Note that AMP components are by default deployed as StatefulSets, which means that storage does not need to be shared between pods. This in turn means that ANY writable storage class will suffice. The Access Mode required is ReadWriteOnce, which is an available mode for all volume plugins.

Provisioner	URL	Notes
Local	https://github.com/rancher/local-path-provisioner	Already installed on RKE2 and Foundation. Easy setup, high performance, but cannot share storage between pods running on different Kubernetes nodes.
In-Cluster NFS Server	https://kubernetes-sigs.github.io/nfs-ganesha-server-and-external-provisioner	If you prefer the ability to share storage across nodes and the convenience of not needing to install your own NFS server, this uses the local provisioner above to create and manage an NFS server inside the cluster.
External NFS Server	https://github.com/kubernetes-sigs/nfs-subdir-external-provisioner	If you prefer to have a centralized NFS server outside of the cluster, then this provisioner can be used.

6.1.5. Installation Bootstrap/Execution Machine

AMP is the ideal location from which to manage deployments. However, a location is needed from which to deploy AMP. This is the Installation Bootstrap/Execution Machine, which has the following requirements:

- Should be a secure machine with limited user access
- Needs a Container Runtime (either Docker OR Podman)

Instructions for installing the required tools can be found at the following locations:

Tool	URL
Podman	https://podman.io/getting-started/
Docker	https://www.docker.com/get-started
Helm	https://helm.sh/docs/intro/install/

6.2. Prepare Helm Chart Configuration

The following table lists the global and amp-specific configurable parameters of AMP chart and their default values.

Parameter	Description	Default
`global.jenkinsHostName`	Default Jenkins host name for ingress	`jenkins.amp.myco.com`
`global.giteaHostName`	Default Gitea host name for ingress	`gitea.amp.myco.com`
`global.nexusHost`	Default Nexus host name for ingress	`nexus.amp.myco.com`
`global.nexusUiHost`	UI Nexus host name for ingress	`ui.nexus.amp.myco.com`

Parameter	Description	Default
`global.nexusDockerPushHost`	Nexus host name for pushing docker images	`docker.push.nexus.amp.myco.com`
`global.nexusDockerPullHost`	Nexus host name for pulling docker images	`docker.pull.nexus.amp.myco.com`
`global.argoCDHostName`	ArgoCD host name for ingress (YAML anchor)	`argocd.amp.myco.com`
`global.ingress.className`	Ingress class name (YAML anchor)	`nginx`
`global.amp.storageClass`	Storage class name (YAML anchor)	`local-path`
`global.certificate.issuer.name`	Global Certificate issuer name (YAML anchor; also used by the cert-manager chart)	`amp-issuer`
`amp.jenkins.enabled`	Flag to enable/disable the installation of the Jenkins component	`true`
`amp.gitea.enabled`	Flag to enable/disable the installation of the Gitea component	`true`
`amp.nexus.enabled`	Flag to enable/disable the installation of the Nexus component	`true`
`amp.certmanager.enabled`	Flag to enable/disable the installation of the cert-manager component	`true`
`amp.argoCD.enabled`	Flag to enable/disable the installation of the Argo CD component	`false`

The following table lists the components configurable parameters with AMP default values.

Parameter	Description	Default
`ge-jenkins.jenkins.persistence.storageClassName` Storage class name used by Jenkins component (YAML anchor reference) `local-path`	`ge-jenkins.jenkins.controller.hostName` Storage class name used by Jenkins component (refers to .Values.global.jenkinsHostName) `jenkins.amp.myco.com`	`ge-jenkins.jenkins.controller.admin.password` Admin password for Jenkins UI `admin123`
`ge-jenkins.jenkins.controller.ingress.ingressClassName` Ingress class name used by Jenkins component (YAML anchor reference) `nginx`	`ge-jenkins.jenkins.controller.ingress.annotations.cert-manager.io/issuer` Certificate issuer name used by Jenkins component (YAML anchor reference) `amp-issuer`	`ge-gitea.persistence.storageClass` Storage class name used by Gitea component (YAML anchor reference) `local-path`
`ge-gitea.gitea.admin.username` Admin usename for Gitea component `root`	`ge-gitea.gitea.admin.password` Admin password for Gitea component `admin123`	`ge-gitea.gitea.ingress.className` Ingress class name used by Gitea component (YAML anchor reference) `nginx`
`ge-gitea.gitea.ingress.annotations.cert-manager.io/issuer` Certificate issuer name used by Gitea component (YAML anchor reference) `amp-issuer`	`ge-nexus.persistence.storageClass` Storage class name used by Nexus component (refers to .Values.global.amp.storageClass) `local-path`	`ge-nexus.ingress.className` Ingress class name used by Nexus component (refers to .Values.global.ingress.className) `nginx`
`ge-nexus.ingress.annotations.cert-manager.io/issuer` Certificate issuer name used by Nexus component (YAML anchor reference) `amp-issuer`	`ge-cert-manager.cert-manager.installCRDs` Enable/disable the installation/uninstallation of cert-manager CRDs `true`	`ge-argo-cd.argo-cd.server.ingress.hostname` Ingress host name used by ArgoCD component (YAML anchor reference) `nginx`

6.3. Install Helm Chart

To install the chart:

```
helm repo add AMP-helm https://dig-grid-artifactory.apps.ge.com/artifactory/virtual-helm
helm install [RELEASE_NAME] AMP --namespace AMP --create-namespace --timeout 20m [flags]
```

The above command deploys AMP on the Kubernetes cluster in the default configuration into the `AMP` namespace. The [configuration](#configuration) section lists the parameters that can be configured during installation.

See [helm install](https://helm.sh/docs/helm/helm_install/) or [helm upgrade](https://helm.sh/docs/helm/helm_upgrade/) for commands documentation.

- the `--timeout` flag is included because some components (such as Nexus) require more time to initialize; the recommended timeout value is at least 20 minutes.
- in addition to the `--timeout` flag, consider using both `--wait` and `--wait-for-jobs` flags.
- when including the `ge-cert-manager` component to your installation of AMP, if the cert-manager CDRs are already existing on your cluster, make sure to set `ge-cert-manager.cert-manager.installCRDs=false` (either with the `--set` flag or into your `values.yaml` file).
- The preferred helm command for installing/upgrading is `helm upgrade [RELEASE_NAME] AMP --install --namespace AMP --create-namespace --timeout 10m [flags]`

7. Uninstalling Chart

```
# Helm 3
$ helm uninstall [RELEASE_NAME] [flags]
```

This removes all the Kubernetes components associated with the chart and deletes the release.

See [helm uninstall](https://helm.sh/docs/helm/helm_uninstall/) for command documentation.

8. Configuration

8.1. Certificates manager

The cert-manager component is provided to bind an issuer to a certificate for SSL/TLS connections to hosts exposed through Ingress, would it be Self-signed, ACME or CA. The type of certificate is configured using the `global.certificate.issuer.type` variable.

8.1.1. CA Issuer

As explained under https://cert-manager.io/docs/configuration/ca/, in order to bind the issuer to a CA certificate and private key, they must be stored into as secrets into the cluster. This can be achieved by following these steps prior to installing AMP:

8.1.1.1. 1. Create secrets using the Cert-Manager pem/crt and key.

```
kubectl create secret tls cm-ca --cert=/path/to/cm-ca.pem --key=/path/to/cm-ca.key -n amp
```

8.1.1.2. 2. Create secret using the Customer root ca.

```
kubectl create secret generic root-ca --from-file=/path/to/root-ca-cert/ -n amp
```

8.1.1.3. 3. Install AMP with the following changes to your *values.yaml*:

- `global.certificate.issuer.type=ca`
- `global.certificate.issuer.ca.secretName=cm-ca` (i.e., the name of the secret holding the CA certificate and private key)

8.1.2. ACME Issuer

As explained under https://cert-manager.io/docs/configuration/acme/#creating-a-basic-acme-issuer, in order to bind the issuer to a Automated Certificate Management Environment (ACME) Certificate Authority server, you will need to configure a clients email, a server URL and a privateKeySecretRef (AMP assumes the use of a HTTP01 solver).

This is done by installing AMP with the following variables in your *values.yaml*:

- `global.certificate.issuer.type=acme`
- `global.certificate.issuer.acme.email` with the Client Email address bound to your account
- `global.certificate.issuer.acme.server` with the Server URL of the ACME certificates provider
- `global.certificate.issuer.acme.privateKeySecretName` with the secret name used to store the account's private key

8.2. Trust Manager

Trust Manager orchestrates bundles of trusted X.509 certificates which are primarily used for validating certificates during a TLS handshake.

Trust-manager is a small Kubernetes operator which aims to help reduce the overhead of managing TLS trust bundles in your clusters. It adds the Bundle custom Kubernetes resource (CRD) which can read input from various sources and combine the resultant certificates into a single bundle ready to be used by your applications.

trust manager works well when consuming CA certificates from a cert-manager Issuer or ClusterIssuer but can be used entirely independently of cert-manager if needed. Trust-manager will keep your trusted certificates up-to-date and enables cluster administrators to easily automate providing a secure bundle.

> The *trust-manager* can be installed along with the *cert-manager* using the parameter `global.trust-manager.enabled=true` as this option is disabled by default.

8.2.1. Bundle

A `Bundle` is a cluster-scoped resource representing a set of X.509 certificates that should be distributed across the cluster. It comprise a list of sources from which trust-manager will assemble the final bundle, along with a target describing how and where the resulting bundle will be placed.

Bundle Structure

```yaml
apiVersion: trust.cert-manager.io/v1alpha1
kind: Bundle
metadata:
  name: my-org.com  # The bundle name will also be used for the target
spec:
  sources:
  # Include a bundle of publicly trusted certificates which can be
  # used to validate most TLS certificates on the internet, such as
  # those issued by Let's Encrypt, Google, Amazon and others.
  - useDefaultCAs: true

  # A Secret in the "trust" namespace; see "Trust Namespace" below for further details
  - secret:
      name: "my-db-tls"
      key: "ca.crt"

  # Here is another Secret source, but this time using a label selector instead of a Secret's name.
  - secret:
      selector:
        matchLabels:
          fruit: apple
      key: "ca.crt"

  # A ConfigMap in the "trust" namespace; see "Trust Namespace" below for further details
  - configMap:
      name: "my-org.net"
      key: "root-certs.pem"

  # Here is another ConfigMap source, but this time using a label selector instead of a ConfigMap's
name.
  - configMap:
      selector:
        matchLabels:
          fruit: apple
      key: "ca.crt"
  target:
    # Sync the bundle to a ConfigMap called `my-org.com` in every namespace which
    # has the label "linkerd.io/inject=enabled"
    # All ConfigMaps will include a PEM-formatted bundle, here named "root-certs.pem"
    # and in this case we also request binary formatted bundles in JKS and PKCS#12 formats,
    # here named "bundle.jks" and "bundle.p12".
    configMap:
      key: "root-certs.pem"
    additionalFormats:
      jks:
        key: "bundle.jks"
        password: 'my-jks-password'
      pkcs12:
        key: "bundle.p12"
        password: 'my-pkcs-password'
    namespaceSelector:
      matchLabels:
        linkerd.io/inject: "enabled"
```

A `Bundle` supports different types of sources (such as configMap, secret or useDefaultCAs) and of targets (i.e. configMap and secret).

All target bundle will be stored as .pem format, but optionally bundle targets can be enabled to have JKS/PKCS#12 formats trust store (as additional formats).

Applications consuming JKS/PKCS#12 trust stores require a password to be set. If none provided, these passwords for trust stores are set by default to *changeit* for JKS and "" (empty string) for PKCS#12. As shown in the example above, these passwords can be overridden.

Target's namespaceSelector is used to restrict which namespaces your `Bundle's target should be synced to.

Please refer to the following content for more details:

- https://cert-manager.io/docs/trust/trust-manager/
- https://cert-manager.io/docs/trust/trust-manager/api-reference/#bundle
- https://cert-manager.io/docs/tutorials/getting-started-with-trust-manager/

Considerations & Limitations

The trust-manager considers 3 namespaces when being installed:

- The trust-manager namespace, which refers to the namespace under which the tool is installed (implicitely, the release namespace).
- The "trust namespace", which refers to the namespace from which the bundle sources will come from, which can be set with parameter `ge-cert-manager.trust-manager.app.trust.namespace`.
- The cert-manager namespace, which is the namespace where cert-manager is installed under where integrating trust-manager with cert-manager (see `ge-cert-manager.trust-manager.app.webhook.tls.approverPolicy.certManagerNamespace` and `ge-cert-manager.trust-manager.app.webhook.tls.approverPolicy.enabled`).

These namespaces can be independent from one another or a single one. Note that:

- for the last two namespaces, the targeted namespace(s) must exist prior to installing *trust-manager*.
- there is a know issue as installations of *trus-manager* with `ge-cert-manager.trust-manager.app.webhook.tls.approverPolicy.enabled=true` may fail. This situation is being investigated and a solution will be provided with a later release.

To support *secret* as the bundle target:

- `ge-cert-manager.trust-manager.secretTargets.enabled` must be enabled, true (default in AMP).
- `ge-cert-manager.trust-manager.secretTargets.authorizedSecretsAll` (default `true` in AMP) or `trust-manager.secretTargets.authorizedSecrets` (default `[]` in AMP) must be set (the second option being more restrictive/secure).

When using *password* with additional formats in the target setting (which overrides the default password), be aware that the password will not be part of the target secret/configMap. This limitation is documented under the issue https://github.com/cert-manager/trust-manager/issues/428.

8.3. User Accounts

Out-the-box AMP uses separate user databases for each component. An administrator account is added for each, and additional accounts can be added as detailed below.

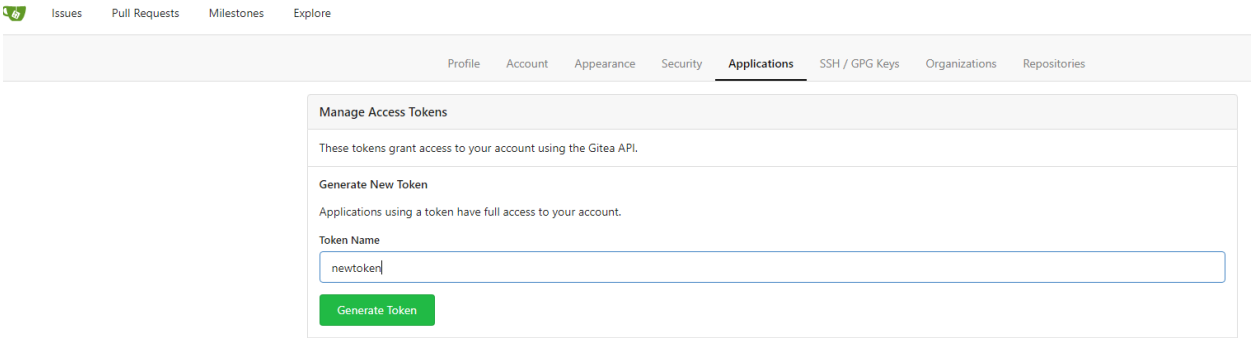
8.3.1. Gitea

The default admin account for Gitea has username `root`, and the password specified during installation. Additional users can be added without need of the admin account by choosing 'Register' from the Gitea landing page:

```
https://<my-host>
```

These new accounts can create organisations and repositories. For any organisation or repository they have created they can add additional members. For general use of Gitea there is no need for administrator intervention or any special actions to create or manage accounts. Users can self-manage, but note that new users cannot invite themselves to existing repositories. Security is ensured by the owners and members of existing organisations and repositories.

If connecting Gitea to Jenkins is required, then there is a need for creating a Token. This is done by clicking on the user profile (right side) → Settings → Applications → add a new name to the token then hit "Generate Token."



After clicking on "Generate token", the token will appear on the screen. Make sure you copy and store it securely, given it will not be shown again. If lost, the only option is to recreate it.

8.3.2. Jenkins

The default admin account for Jenkins has username `admin`, and the password specified during installation. Additional users must be explicitly added by an existing administrator.

With this default configuration only a single user exists. Although the manifest does allow additional users to be added, this is an inconvenient way to add users going forward. The simplest approach is to use Jenkins' own database.

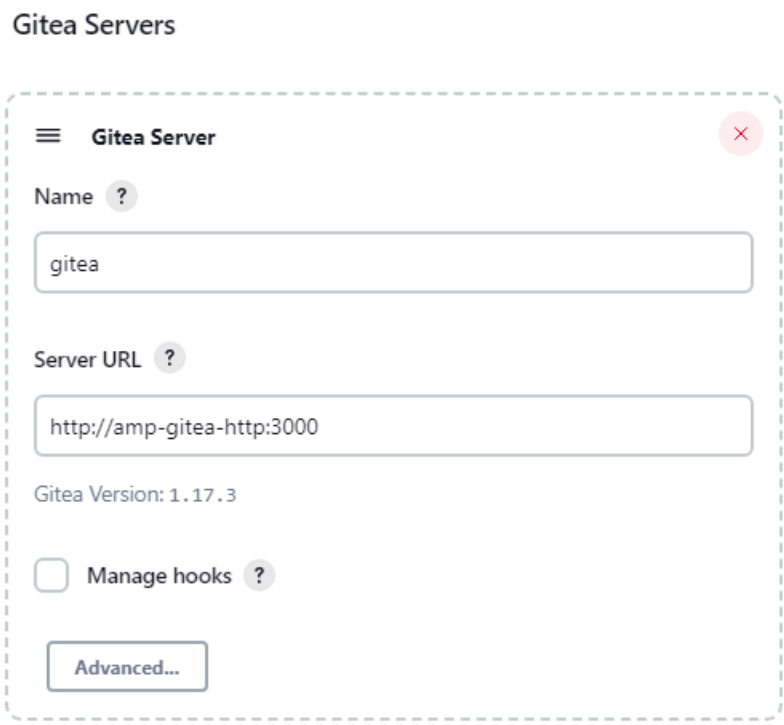
To do this select 'Manage Jenkins' from the Jenkins landing page:

```
https://jenkins.<my-host>/manage
```

Choose 'Configure Global Security' and set the Security Realm to be Jenkins' Own Database. You will then be invited to create the first admin user. Note that activating this will DISABLE the initial admin user account.

Once this account has been created additional users can be added by selecting 'Manage Jenkins' and then 'Manage Users' from the Jenkins Landing Page. A 'Create User' option will be available to the left.

For adding Gitea as a source, navigate to "Manage Jenkins" → "Configure System" → "Gitea Servers" → Click "Add" → "Gitea Server". Put a recognizable name (for example, Gitea) and under Server URL specify the service name image. After performing this change, by clicking anywhere you should get the URL validated as below:



8.3.3. Nexus

The default admin account for the Nexus has username `admin`, and the password specified during installation. Additional users must be explicitly added by an existing administrator.

Once logged in to the Nexus select the 'Server Administration and Configuration' cog from the top of the UI. Choose 'Users' from the 'Security' group to the left, and click on the 'Create local user' button.

Note that the out-the-box configuration allows anonymous download from the Nexus. This can be changed by selecting 'Anonymous Access', after selecting the 'Server Administration and Configuration' cog.

8.3.4. LDAP and SAML Authentication

All four components support the use of an external LDAP server, and Jenkins and Nexus support SAML. Instructions for configuring this can be found at:

Component	Technology	URL
Nexus Repository Manager	LDAP	https://help.sonatype.com/repomanager3/system-configuration/user-authentication/ldap
Gitea	LDAP	https://docs.gitea.io/en-us/authentication/#ldap-lightweight-directory-access-protocol
Jenkins	LDAP	https://plugins.jenkins.io/ldap/
Nexus Repository Manager	SAML	https://help.sonatype.com/repomanager3/system-configuration/user-authentication/saml

Component	Technology	URL
Jenkins	SAML	https://github.com/jenkinsci/saml-plugin/blob/master/doc/CONFIGURE.md

8.4. Trusting Certificates in AMP Components

8.4.1. Using a secret to store certificates

In the case certificates shall be provided to the AMP components (i.e. the controller and agents) in order to be trusted for secure integrations, the configurations per components presented below assume a Kubernetes secret to be created into the namespace.

This secret shall be holding a list of `.crt` files in its data, each one being a certificate to be trusted (in a PEM format). The secret definition would look something like (with one or many files listed):

```
apiVersion: v1
kind: Secret
metadata:
  name: amp-trusted-ca-secret
data:
  tls.crt: < tls.crt file content as base64 >
  ca.crt: < ca.crt file content as base64 >
  ...
  another-ca.crt: < another-ca.crt file content as base64 >
```

8.4.1.1. Jenkins Controller

To have certificates trusted by the Jenkins controller for secure integrations (e.g. a LDAPS or Active Directory), we can achieve it by configuring the ge-jenkins section of the AMP `values.yaml` as shown below:

```yaml
ge-jenkins:
  jenkins:
    controller:
      # Setting the Jenkins controller Java properties to use the mounted Java truststore created in
the initContainer
      javaOpts: "-Djavax.net.ssl.trustStore=/opt/java/openjdk/lib/security/cacerts
-Djavax.net.ssl.trustStorePassword=changeit"
      # Jenkins controller initContainer for creating the new keystore
      customInitContainers:
        - name: additional-ca-certs
          image: '{{ .Values.controller.image.registry }}/{{ .Values.controller.image.repository
}}:{{- .Values.controller.image.tag -}}'
          imagePullPolicy: IfNotPresent
          securityContext:
            allowPrivilegeEscalation: true
            runAsUser: 0
            runAsNonRoot: false
          command:
            - sh
          args:
            - -c
            - |
              # Copy existing JVM `cacerts` file and
              cp ${JAVA_HOME}/lib/security/cacerts /ca-certificates/cacerts
              # Add all additional ca certs
              for file in /tmp/additional-ca-certs/*.crt; do
                filename=$(basename -- "$file" .crt)
                ${JAVA_HOME}/bin/keytool -import -trustcacerts -noprompt -keystore /ca-
certificates/cacerts -storepass changeit -alias "amp-$filename" -file "$file"
              done
          volumeMounts:
            - name: additional-ca-certs
              mountPath: /tmp/additional-ca-certs
            - name: amp-tls-ca-truststore
              mountPath: /ca-certificates
    persistence:
      volumes:
        - name: amp-tls-ca-truststore
          emptyDir: {}
        - name: additional-ca-certs
          secret:
            secretName: amp-trusted-ca-secret
      mounts:
        - mountPath: /opt/java/openjdk/lib/security/cacerts
          name: amp-tls-ca-truststore
          subPath: cacerts
```

The sample above is responsible for:

- Set and mount the volumes needed by the Jenkins pod using the `jenkins.persistence.volumes` and `jenkins.persistence.mounts` variable
- Prepare the Java TrustStore using the `keytool` java utility and the content of the secret (mounted

as a volume) by means of an initContainer (set under `jenkins.controller.customInitContainers`) with the controller image and a bash script and storing as the `cacerts` file into another volume.

- With `jenkins.controller.javaOpts`, the JVM is set to use the so-created `cacerts` file as the default trustStore for SSL\TLS validation.

8.4.1.2. Jenkins Agent

As the configuration of the *default* agent pod relies on the podTemplate set into the *kubernetes* Jenkins plugin, the certificates shall be inputted by injecting the proper volumes, mounts and initContainers in the 'Raw YAML for the Pod' so it can be merged with the rest of the agent pod configuration attributes. To do so, the ge-jenkins `values.yaml` shall be configured with the following parameters:

```
ge-jenkins:
  jenkins:
    agent:
      yamlMergeStrategy: "merge"
      yamlTemplate: |-
        spec:
          containers:
            - name: jnlp
              volumeMounts:
              - mountPath: "/etc/pki/tls/certs/ca-bundle.crt"
                name: amp-tls-ca-bundle
                subPath: tls-ca-bundle.pem
                readOnly: true
          initContainers:
            - name: additional-ca-certs
              image: '{{ .Values.agent.image.repository }}:{{ .Values.agent.image.tag }}'
              imagePullPolicy: IfNotPresent
              securityContext:
                allowPrivilegeEscalation: true
                runAsUser: 0
                runAsNonRoot: false
              command:
              - sh
              args:
              - -c
              - |
                # Copy existing tls-ca-bundle.pem file from image (if any)
                [[ -f "/etc/pki/ca-trust/extracted/pem/tls-ca-bundle.pem" ]] && cat /etc/pki/ca-
trust/extracted/pem/tls-ca-bundle.pem > /ca-certificates/tls-ca-bundle.pem
                # Add all additional ca certs
                for file in /tmp/additional-ca-certs/*.crt; do
                  filename=$(basename -- "$file" .crt)
                  echo "# amp-$filename" >> /ca-certificates/tls-ca-bundle.pem
                  cat  $file >> /ca-certificates/tls-ca-bundle.pem
                  echo "" >> /ca-certificates/tls-ca-bundle.pem
                done
              securityContext:
                privileged: true
                runAsUser: 0
              volumeMounts:
              - mountPath: /tmp/additional-ca-certs
                name: additional-ca-certs
              - mountPath: /ca-certificates
                name: amp-tls-ca-bundle
          volumes:
            - name: amp-tls-ca-bundle
              emptyDir: {}
            - name: additional-ca-certs
              secret:
                secretName: amp-trusted-ca-secret
```

The sample above is responsible for:

- Enforce the merging of Raw YAML for the agent Pod with other pod settings by using `jenkins.agent.yamlMergeStrategy=merge`.
- Through the use of `jenkins.agent.yamlTemplate`, the pod base specification is provided:
- Set and mount the volumes needed by the Jenkins agent pod under `spec.volumes`.
- Prepare the CA bundle file using the content of the secret (mounted as a volume) by means of an initContainer (under `spec.initContainers`, container named 'additional-ca-certs') with the agent image and a bash script bundling as the `tls-ca-bundle.pem` file into a dedicated volume.
- Under `spec.containers`, injecting into the 'jnlp' container an extra volume mount so the content of generated `tls-ca-bundle.pem` file is used as the system/container `/etc/pki/tls/certs/ca-bundle.crt`.

8.4.1.3. Nexus Repository Server

To have certificates trusted by the Nexus Repository for secure integrations (e.g. a LDAPS or Active Directory), we can achieve it by configuring the ge-nexus section of the AMP `values.yaml` as shown below:

```
ge-nexus:
  deployment:
    initContainers:
      - name: additional-ca-certs
        image: '{{ .Values.nexus.image.repository }}:{{- .Values.nexus.image.tag -}}'
        imagePullPolicy: IfNotPresent
        securityContext:
          allowPrivilegeEscalation: true
          runAsUser: 0
          runAsNonRoot: false
        command:
          - sh
        args:
          - -c
          - |
            # copy /opt/sonatype/nexus/bin/nexus.vmoptions
            cp /opt/sonatype/nexus/bin/nexus.vmoptions /ca-certificates/my-nexus.vmoptions
            # copy existing JVM `cacerts` file and
            cp ${JAVA_HOME}/lib/security/cacerts /ca-certificates/cacerts/etc/pki/ca-
trust/extracted/pem/tls-ca-bundle.pem > /ca-certificates/tls-ca-bundle.pem
            # add all additional ca certs
            for file in /tmp/additional-ca-certs/*.crt; do
              filename=$(basename -- "$file" .crt)
              ${JAVA_HOME}/bin/keytool -import -trustcacerts -noprompt -keystore /ca-
certificates/cacerts -storepass changeit -alias "amp-$filename" -file "$file"
            done
            # Add SSL Java options to nexus.vmoptions
            echo "-Djavax.net.ssl.trustStore=/nexus-data/custom-truststore.jks" >> /ca-
certificates/my-nexus.vmoptions
            echo "-Djavax.net.ssl.trustStorePassword=changeit" >> /ca-certificates/my-nexus.vmoptions
            cat /ca-certificates/my-nexus.vmoptions
        volumeMounts:
          - name: additional-ca-certs
            mountPath: /tmp/additional-ca-certs
          - name: amp-tls-ca-truststore
            mountPath: /ca-certificates
    additionalVolumes:
      - name: amp-tls-ca-truststore
        emptyDir: {}
      - name: additional-ca-certs
        secret:
          secretName: amp-trusted-ca-secret
    additionalVolumeMounts:
      - name: amp-tls-ca-truststore
        mountPath: "/nexus-data/custom-truststore.jks"
        subPath: cacerts
        readOnly: true
      - name: amp-tls-ca-truststore
        mountPath: "/opt/sonatype/nexus/bin/nexus.vmoptions"
        subPath: my-nexus.vmoptions
        readOnly: true
```

The sample above is responsible for:

- Set the volumes needed by the Nexus pod using the `deployment.additionalVolumes` variable
- By means of an initContainer (set under `deployment.initContainers`) and the content of the secret (mounted as a volume) with the nexus image and a bash script:
- Prepare the Java TrustStore using the `keytool` java utility and the content of the secret (mounted as a volume) and storing as the `cacerts` file into another volume.
- Prepare the custom `nexus.vmoptions` (based on the default) and adding the Java options to use the trustStore.
- Mount volumes to the Nexus pod main container using `deployment.additionalVolumeMounts` variable so the nexus application and the JVM use the so-created nexus.vmoptions and custom-truststore.jks.

8.4.2. Using a trust-manager bundle

Assuming that trust-manager is installed, which is a tool allowing to easily manage and distribute certificates to be trusted into a cluster (see section Trust Manager above):

- Trust-manager basically can be used to manage the certificates and automatically distribute them to multiple namespaces.
- It is responsible for managing and validating the trust relationships between certificates.
- It validates the expiry of the certificate and has been signed by trusted CA.
- Also, Ensruing that a certificate chains up to a trusted root CA, which implies that all intermediate certificates in the chain are valid and trusted.

With trust-manager installed on the cluster where the AMP deployment resides, we shall use a `Bundle` to provide the certificates to be trusted by the AMP components in an unified manner.

The example below shows a `Bundle` taking certificates to be trusted from different sources and creating a bundle as a secret into the targeted namespace in a PEM and JKS format.

As you have already created a `Bundle` and a `Secret` holding the keystore password have been created as presented above(During the nexus configuration), Nexus can bind to them using the following configuration parameters into the AMP `values.yaml` files - Important fields to be provided. `JksSecretNam:` Provide the name of the secret created along with the bundle<bundle=example-bundle>. `JksSecretKey:` Provide the jks file name<bundle=example-bundle.jks>. `JksPasswordSecretName:` Provide the secret name created to secure the jks password<secret-name>. `JksPasswordSecretKey:` Provide the key which consists of jks password<key>.

```yaml
apiVersion: trust.cert-manager.io/v1alpha1
kind: Bundle
metadata:
  name: amp-ca-bundle
spec:
  sources:
  - useDefaultCAs: true
  - secret:
      name: "some-tls-secret"
      key: "ca.crt"
  - configMap:
      name: "my-org.net"
      key: "root-certs.pem"
  - configMap:
      selector:
        matchLabels:
          fruit: apple
      key: "ca.crt"
  target:
    secret:
      key: "trust-bundle.pem"
    additionalFormats:
      jks:
        key: "trust-ca-bundle.jks"
        password: "some-jks-password"
    namespaceSelector:
      matchLabels:
        kubernetes.io/metadata.name: amp
```

Once the `Bundle` is configured, trust-manager will automatically generate a `Secret` into the targeted namespace(s) and having for name the one of the Bundle. This secret should contain items with the key(s) defined into the Bundle target.

- For more details on the `Bundle` API, please refer to https://cert-manager.io/docs/trust/trust-manager/api-reference/#bundle
- The generated secret will have the name of the bundle (i.e. 'amp-ca-bundle' in this example).
- As of the limitation for using *password* with additional formats in the target setting (see issue https://github.com/cert-manager/trust-manager/issues/428), this information will not be part of the target secret .
  - To mitigate that limitation, another secret with this piece of information shall be created in the targeted namespace(s) so it can be referred accordingly where needed. In the following sub-sections, we shall assume a secret named 'my-trust-secret' with the password under the key 'my-jks-password'.

8.4.2.1. Jenkins Controller

Assuming that a `Bundle` and a `Secret` holding the keystore password have been created as in the

Bundle example above, the Jenkins controller can bind to them using the following configuration parameters into the AMP `values.yaml` files.

```
ge-jenkins:
  jenkins:
    controller:
      javaOpts: "-Djavax.net.ssl.trustStore=/opt/java/openjdk/lib/security/cacerts
-Djavax.net.ssl.trustStorePassword=$(JKS_PASSWORD)"
      containerEnv:
      - name: JKS_PASSWORD
        valueFrom:
          secretKeyRef:
            name: my-trust-secret
            key: my-jks-password
    persistence:
      volumes:
      - name: trust-ca-bundle
        secret:
          secretName: amp-ca-bundle
      mounts:
      - name: trust-ca-bundle
        mountPath: /opt/java/openjdk/lib/security/cacerts
        subPath: 'trust-ca-bundle.jks'
      - mountPath: "/etc/pki/tls/certs/ca-bundle.crt"
        name: trust-ca-bundle
        subPath: trust-bundle.pem
        readOnly: true
```

The sample above is responsible for:

- Set a volume to bind to the `Secret` generated by the trust-manager using `jenkins.persistence.volumes`
- Mount volumes to the Jenkins pod to inject the JKS as well as the PEM certificates bundle from the secret using `jenkins.persistence.mounts`

8.4.2.2. Jenkins Agent

Assuming that a `Bundle` has been created, the Jenkins agent can bind to the PEM certificates bundle from the secret by configuring the following into the AMP `values.yaml`:

```yaml
ge-jenkins:
  jenkins:
    agent:
      yamlMergeStrategy: "merge"
      yamlTemplate: |-
        spec:
          containers:
            - name: jnlp
              volumeMounts:
              - mountPath: /etc/pki/tls/certs/ca-bundle.crt
                name: trust-ca-bundle
                subPath: trust-bundle.pem
                readOnly: true
          volumes:
            - name: trust-ca-bundle
              secret:
                secretName: amp-ca-bundle
```

The sample above is responsible for:

- Enforce the merging of Raw YAML for the agent Pod with other pod settings by using `jenkins.agent.yamlMergeStrategy=merge`.
- Through the use of `jenkins.agent.yamlTemplate`, the pod base specification is provided:
  - Set the volume needed by the Jenkins agent pod under `spec.volumes`.
  - Under `spec.containers`, injecting into the 'jnlp' container an extra volume mount so the content of generated `trust-bundle.pem` file is used as the system/container `/etc/pki/tls/certs/ca-bundle.crt`.

8.4.2.3. Nexus Repository Server

Assuming that a trust-manager Bundle has been set to generate a certificates bundle secret and a Secret holding the keystore password have been created (both secrets found in the component namespace), Nexus can bind to them using the following configuration parameters into the AMP values.yaml files as shown below:

- `JksSecretName`: Provide the name of the secret holding the JKS file.
- `JksSecretKey`: Provide the key the JKS is under.
- `JksPasswordSecretName`: Provide the secret name holding the JKS file password.
- `JksPasswordSecretKey`: Provide the key of the JKS file password.

```yaml
ge-nexus:
  deployment:
    trustStore:
      JksSecretName: 'amp-ca-bundle'
      JksSecretKey: 'trust-ca-bundle.jks'
      JksPasswordSecretName: 'my-trust-secret'
      JksPasswordSecretKey: 'my-jks-password'
```

8.4.2.4. Gitea

Assuming that the Trust-manager `Bundle` has been set to generate a certificate bundle secret, Gitea can bind to bundled certificates using the following configuration parameters into the AMP values.yaml file as shown below:

```yaml
ge-gitea:
  gitea:
    extraVolumeMounts:
      - name: trust-test
        mountPath: /etc/ssl/certs/ca-bundle.crt
        subPath: trust-bundle.pem
        readOnly: true
    extraVolumes:
      - name: trust-test
        secret:
          secretName: amp-ca-bundle
```

The sample above is responsible for:

- Set a volume to bind to the `Secret` generated by the trust-manager using `ge-gitea.gitea.extraVolumes`
- Mount volumes to the gitea pod to inject the PEM certificates bundle from the secret using `ge-gitea.gitea.extraVolumeMounts`

8.4.2.5. Argo CD

Assuming that a trust-manager `Bundle` has been created (see introduction of Using a trust-manager bundle), the Argo CD can bind to the PEM certificates bundle from the generated secret by configuring the following into the AMP `values.yaml`:

```yaml
ge-argo-cd:
  argo-cd:
    repoServer:
      volumes:
        - name: trusted-ca-certificates
          secret:
            secretName: amp-ca-bundle
      volumeMounts:
        - name: trusted-ca-certificates
          mountPath: /etc/ssl/certs/ca-certificates.crt
          subPath: trust-bundle.pem
          readOnly: true
    server:
      volumes:
        - name: trusted-ca-certificates
          secret:
            secretName: amp-ca-bundle
      volumeMounts:
        - name: trusted-ca-certificates
          mountPath: /etc/ssl/certs/ca-certificates.crt
          subPath: trust-bundle.pem
          readOnly: true
    controller:
      volumes:
        - name: trusted-ca-certificates
          secret:
            secretName: amp-ca-bundle
      volumeMounts:
        - name: trusted-ca-certificates
          mountPath: /etc/ssl/certs/ca-certificates.crt
          subPath: trust-bundle.pem
          readOnly: true
    dex:
      volumes:
        - name: trusted-ca-certificates
          secret:
            secretName: amp-ca-bundle
      volumeMounts:
        - name: trusted-ca-certificates
          mountPath: /etc/ssl/certs/ca-certificates.crt
          subPath: trust-bundle.pem
          readOnly: true
    redis:
      volumes:
        - name: trusted-ca-certificates
          secret:
            secretName: amp-ca-bundle
      volumeMounts:
        - name: trusted-ca-certificates
          mountPath: /etc/ssl/certs/ca-certificates.crt
          subPath: trust-bundle.pem
          readOnly: true
```

These values do the following for each of the ArgoCD components (server, controller, repoServer, dex, redis):

- Set a volume to bind to the `Secret` generated by the trust-manager using `volumes` for the ArgoCD component.
- Mount the volume to the component pod to inject the PEM certificates using `volumeMounts`.

8.5. Using Secrets with Default/Admin Users

In order to avoid setting as plain text sensitive information such as default password, AMP components support having such details into kubernetes secrets. What follows presents such configuration per components.

8.5.1. Jenkins Admin User credentials into a Kubernetes Secret

In order to avoid using plain text values for setting the Jenkins Admin user credentials, these pieces of information can be stored in a `Secret` resource into the kubernetes cluster.

Assuming a secret in the following format:

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: jenkins-admin-credentials
  namespace: amp
type: Opaque
data:
  jenkins-admin-user: <base64-encoded-username>
  jenkins-admin-password: <base64-encoded-password>
```

The Jenkins deployment can be configured to use it by applying to the *AMP* `values.yaml` the parameters shown below:

```yaml
ge-jenkins:
  jenkins:
    controller:
      admin:
        createSecret: true
        existingSecret: jenkins-admin-credentials  # Reference to the existing secret
        userKey: jenkins-admin-user            # Key for the username in the secret
        passwordKey: jenkins-admin-password    # Key for the password in the secret
```

Any changes applied to the Secret values will only be picked up when the Jenkins pod is restarted afterward.

8.5.2. Gitea Admin User credentials into a Kubernetes Secret

In order to avoid using plain text values for setting the Gitea Admin user credentials, these pieces of information can be stored in a `Secret` resource into the kubernetes cluster.

Assuming a secret in the following format:

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: gitea-admin-secret
  namespace: amp
type: Opaque
data:
  username: <base64-encoded-username>
  password: <base64-encoded-password>
```

We must define only keywords of "username" and "password" in the secret in the context of Gitea k8s secrets management.

The Gitea deployment can be configured to use it by applying to the *ge-gitea* `values.yaml` the parameters shown below:

```yaml
ge-gitea:
  gitea:
    gitea:
      admin:
        existingSecret: gitea-admin-secret  # name of the secret which created to store gitea
credentials.
```

Any changes applied to the Secret values will only be picked up when the gitea pod is restarted afterward.

8.5.3. ArgoCD Admin User credentials into a Kubernetes Secret

In order to avoid using plain text values for setting the ArgoCD Admin user credentials, these pieces of information can be stored in a `argocd-secret` resource into the kubernetes cluster.

Assuming a secret in the following format:

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: argocd-secret
  namespace: amp
  labels:
    app.kubernetes.io/name: argocd-secret
    app.kubernetes.io/part-of: argocd
type: Opaque
data:
    # -- Bcrypt hashed admin password
    ## Argo expects the password in the secret to be bcrypt hashed. You can create this hash with
    ## `htpasswd -nbBC 10 "" $ARGO_PWD | tr -d ':\n' | sed 's/$2y/$2a/'`
  admin.password: '<argocd-password-base64-encoded>'
  admin.passwordMtime:
  server.secretkey:
```

The `admin.passwordMtime` and `server.secretkey` key that are required into the secret, but with empty values as the ArgoCD installation will populate them afterward.

The argocd deployment can be configured to use it by applying to the *argo-cd* `values.yaml` the parameters shown below:

```yaml
ge-argo-cd:
  argo-cd:
    configs:
      secret:
        createSecret: false
```

Any changes applied to the Secret values will only be picked up when the `argocd-server pod` is restarted afterward.

8.5.4. Nexus Admin User password into a Kubernetes Secret

In order to avoid using plain text values for setting Nexus initial Admin. password, this information can be stored in a `Secret` resource into the kubernetes cluster.

Assuming a secret in the following format:

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: nexus-admin-init-creds
  namespace: amp
type: Opaque
data:
  nexus-init-password: <base64-encoded-password>
```

The Nexus deployment can be configured to use it by applying to the *AMP* `values.yaml` the parameters shown below:

```yaml
ge-nexus:
  initAdminPassword:
    enabled: true
    # Using a Secret for the Init Admin Password
    secret:
      existingSecret: "nexus-admin-init-creds" # The secret name holding the Init Admin Password
      passwordKey: "nexus-init-password" # The key in the secret holding the Init Admin Password
```

This password initialization will only work on fresh AMP installation and not on an update/upgrade. This is due to the fact that the Nexus application is persisting the initial password in its own storage, which does not synchronize with changes in the secret.

9. Airgap Installation

The current bundle is a package holding all the artifacts needed for installing the Automation Management Platform (AMP) into an airgapped environment. AMP installation assumes a single-node cluster.

9.1. Prerequisites

9.1.1. Bootstrap/Execution machine:

- Linux machine (Recommanded RHEL)
- `podman` or `docker` installed
- SSH client installed (e.g., on RHEL `yum install openssh-clients -y` or `dnf install openssh-clients -y`)

9.1.2. Target machine(s):

- Linux machine RHEL/CentOS version 8.3 or above
- Linux machine components updated to their latest versions (i.e., `yum update -y` or `dnf update -y`)
- *container-selinux* package installed (i.e., `yum install container-selinux -y` or `dnf install container-selinux -y`)
- If using the all-in-one bundle to install the RKE2 cluster:
  - Installation on RHEL 9.x is not yet supported
  - no `docker` installed
  - SELINUX may need to be set to *permissive* (`vi /etc/selinux/config` and configure `SELINUX=permissive`)
- Requirements:
  - a minimum of 4 (v)CPU's
  - a minimum of 8GB of RAM
  - a minimum of 80GB of storage
  - Inbound Rules - Ports:
    - 22: SSH/SCP connection for all pre-installation steps to reach the target node(s).
    - 443: HTTPS connection for web traffic.
    - 6443: Connection for web Kubernetes API for deployment (via `helm` or `kubectl` CLI).

9.2. Instructions

The installation considers 3 steps:

1. AMP Airgap installation preparation (with the `airgap-pre-installation.sh` script)

2. AMP installation (with Helm)
3. AMP Airgap post-installation (with the `airgap-post-installation.sh` script)

9.2.1. Step #1: AMP Airgap installation preparation

The `airgap-pre-installation.sh` script is meant to ease the pre-installation (such as loading the required images on the target node) of the Automation Management Platform (AMP) with minimal interaction.

- When using the all-in-one bundle, this script also allows the installation of an RKE2 cluster on which to install AMP.
  - The installation of RKE2 may take 30-35 minutes (or more) to finalize based on your environment.
  - If using `podman` from the Bootstrap machine, for airgap all-in-one bundle up to AMP 3.2.0, you may need to manually start the podman service in your session prior to runnung the script:
    - `podman system service -t 0 &`
    - `export DOCKER_HOST=unix:/var/run/podman/podman.sock`

At minimum, the pre-installation command will look like:

```
./airgap-pre-installation.sh --ip <target-ip> --key /path/to/ssh.pem --user <target-user>
```

For full details of its usage, use command: `./airgap-pre-installation.sh --help`

Starting with AMP 3.2.1 all-in-one bundle, the RKE2 installation has changed to a non-PDI one. Our tests for upgrading the RKE2 installation from the AMP 3.2.0 all-in-one bundle (PDI based) to AMP 3.2.1 all-in-one bundle one have been successful and without data loss. Nonetheless, we cannot guarantee it is safe regardless the amount of data in your AMP installation, hence we recommend that you backup your cluster prior to applying such upgrade.

Docker images included into the AMP airgap bundles

- dig-grid-artifactory.apps.ge.com/virtual-docker/acid/logical-backup:v1.15.0
- dig-grid-artifactory.apps.ge.com/virtual-docker/acid/pgbouncer:master-26
- dig-grid-artifactory.apps.ge.com/virtual-docker/argoproj/argocd:v3.1.9
- dig-grid-artifactory.apps.ge.com/virtual-docker/argoprojlabs/argocd-extension-installer:v0.0.8
- dig-grid-artifactory.apps.ge.com/virtual-docker/bats/bats:1.12.0
- dig-grid-artifactory.apps.ge.com/virtual-docker/bitnami/redis-exporter:1.76.0
- dig-grid-artifactory.apps.ge.com/virtual-docker/bsf/amp-nexus:3.85.0.build1
- dig-grid-artifactory.apps.ge.com/virtual-docker/bsf/gitea/gitea:1.24.7.build1-rootless
- dig-grid-artifactory.apps.ge.com/virtual-docker/bsf/jenkins-controller-lite:2.535
- dig-grid-artifactory.apps.ge.com/virtual-docker/bsf/jenkins-pink-agent:3345.v03dee9b_f88fc-8.0.18
- dig-grid-artifactory.apps.ge.com/virtual-docker/bsf/kiwigrid-k8s-sidecar:1.30.10
- dig-grid-artifactory.apps.ge.com/virtual-docker/bsf/python-test:U1-201P3.9-build.46
- dig-grid-artifactory.apps.ge.com/virtual-docker/curlimages/curl:8.16.0
- dig-grid-artifactory.apps.ge.com/virtual-docker/dexidp/dex:v2.44.0
- dig-grid-artifactory.apps.ge.com/virtual-docker/jetstack/cert-manager-acmesolver:v1.19.1
- dig-grid-artifactory.apps.ge.com/virtual-docker/jetstack/cert-manager-cainjector:v1.19.1
- dig-grid-artifactory.apps.ge.com/virtual-docker/jetstack/cert-manager-controller:v1.19.1
- dig-grid-artifactory.apps.ge.com/virtual-docker/jetstack/cert-manager-package-debian:20210119.2
- dig-grid-artifactory.apps.ge.com/virtual-docker/jetstack/cert-manager-startupapicheck:v1.19.1
- dig-grid-artifactory.apps.ge.com/virtual-docker/jetstack/cert-manager-webhook:v1.19.1
- dig-grid-artifactory.apps.ge.com/virtual-docker/jetstack/trust-manager:v0.20.2
- dig-grid-artifactory.apps.ge.com/virtual-docker/library/busybox:1.37.0
- dig-grid-artifactory.apps.ge.com/virtual-docker/library/redis:8.2.2-alpine
- dig-grid-artifactory.apps.ge.com/virtual-docker/postgres-operator:1.16.0-6035fdd-build.25
- dig-grid-artifactory.apps.ge.com/virtual-docker/spilo-cdp:17-4.0-p2-build.21

When using the all-in-one bundle, after installing the RKE2 cluster, make sure to revert SELINUX as per your security policy on the target machine(s) (`vi /etc/selinux/config` and configure `SELINUX` accordingly)

9.2.2. Step #2: AMP installation

This step relies on the outcome from the previous step:

- if using the all-in-one bundle and the RKE2 cluster got installed, it recovers a copy of the K8s configuration file to connect to it.
- it prepares a `value-overrides.yaml` adapted to the current AMP airgap installation.

The installation is done using helm with the chart provided and the `value-overrides.yaml` prepared:

```
./bin/helm upgrade [RELEASE_NAME] AMP-*.tgz --install -f value-overrides.yaml --namespace amp --create
-namespace --wait --wait-for-jobs --timeout 25m  [flags]
```

- Because the `helm upgrade` command allows the use of multiple `--value`/`-f` flags, you can take advantage of it by providing the bundle-provided configuration values file (i.e. `value-overrides.yaml`) as well as another one prepared with your solution customizations (e.g. `solution-value-overrides.yaml`). The command will look like: `./bin/helm [RELEASE_NAME] amp AMP-*.tgz --install -n amp --create -namespace -f value-overrides.yaml -f solution-value-overrides.yaml`
- If at step #1 you used the registry population options from the `airgap-pre-installation.sh` script, the helm command will be using the `value-overrides-rewritten.yaml` file instead of `value-overrides.yaml`.

For further information on configuring and installing AMP, please refer to its installation guide.

9.2.3. Step #3: AMP Airgap post-installation (optional)

The `airgap-post-installation.sh` script is a utility script allowing to push the AMP images loaded int the target node image store (at step 1) into the image registry provided by Nexus from the AMP installation.

For full details of its usage, use command: `sh airgap-post-installation.sh --help`

If during the AMP installation you used an additional configuration values file (e.g. `solution-value-overrides.yaml`), make sure to append `-f solution-value-overrides.yaml` to the helm command displayed by the `airgap-post-installation.sh` execution.

10. Upgrade

10.1. Upgrade Documentation

Future releases of AMP will come with a section on Upgrading in the Software Installation and Maintenance Guide. Any specific notes for moving between versions will be included.

However, the upgrade process should in general be identical to the installation process. The process is described below.

10.2. Overview

The Automation Management Platform is supplied as a Manifest, describing the desired state of the deployment. Upgrading is essentially just telling the system what the new desired state is.

To achieve this there are four steps:

1. Ensure the system can download the required Images
2. Retrieve the latest AMP helm chart
3. Reconcile your own *values.yaml* changes with the upgraded one (if needed)
4. Upgrade the existing installation with the helm chart

10.3. Image Availability

The functional components of AMP run in containers, downloaded as Images. If your infrastructure has access to GE Digital's Grid Artifactory (https://dig-grid-artifactory.apps.ge.com/) then no work is required - the new Images required for the upgrade will be automatically available.

Where downloading directly from Grid Artifactory is not possible then the 'Cache Container Process' must be used to install images into your local registry. Please follow the instructions in the 'Bare-Metal Manifest'

10.4. Prepare AMP Installation Upgrade

When upgrading the helm-based AMP, we must make sure to:

- Use the proper configuration into the *values.yaml* specific to your current installation
  - Have it aligned to the upgrade intended.
- Ensure breaking changes are addressed accordingly (see Breaking Changes)
  - Breaking changes may be items of configuration (in *values.yaml*) or of other nature. Review them careful before upgrading to a newer version of AMP.
- Use the latest version AMP Helm Chart

We recommend using a Source Control system (Git) for maintaining your *values.yaml* files, which assists with 'three-way merge' of changing files. The goal is to ensure that the changes made to the original file are re-applied to the new upgrade one.

10.5. Apply The AMP upgrade

The 'helm upgrade' command shall be used used to upgrade AMP. Ensure the updated *values.yaml* is in place and type:

```
helm upgrade [RELEASE] AMP-*.tgz -n [NAMESPACE] --timeout 15m
```

Use the `--values` flag with your deployment *values.yaml*

Known Issues

After upgrade:

- Gitea may ask you to update your admin user password on first login after the upgrade.

It is recommended to run this command from the same environment used for the initial installation (the 'Installation Bootstrap\Execution Machine' described in the Installation section). If during the initial installation of AMP the '--kubeconfig' flag was used, this flag with the same kube configuration file must be used.

11. Breaking Changes

11.1. AMP 4.1.1 (and argo_cd_v3.0.11)

- Fine-grained RBAC inheritance for application sub-resources is now disabled by default. You must update policies to include `/update/` and `/delete/` or revert behavior using `server.rbac.disableApplicationFineGrainedRBACInheritance`.
- Pod log access now requires explicit RBAC permission (`logs, get`) as `server.rbac.log.enforce.enable` has been removed.
- Default resource tracking is now annotation-based instead of label-based. Set `application.resourceTrackingMethod: label` to retain legacy behavior.
- Legacy metrics (`argocd_app_sync_status`, `argocd_app_health_status`, etc.) have been removed. Use `argocd_app_info` metric with labels instead.

For more information please refer to https://argo-cd.readthedocs.io/en/stable/operator-manual/upgrading/2.14-3.0/

11.2. AMP 4.1.0 (and *ge-gitea* 12.0.0)

- Feature of "Configuring Actions" has been removed from the gitea helm-chart, for more information please refer to https://gitea.com/gitea/helm-gitea/releases/tag/v12.0.0

11.3. AMP 4.0.0 (and *ge-nexus* 2.0.0): Sonatype Nexus Repository, OrientDB End Of Support

Starting with Nexus 3.71.0, which relies on Java 17, data storing using OrientDB is no more supported; please refer to https://help.sonatype.com/en/sonatype-nexus-repository-3-71-0-release-notes.html for more details.

In order to take advantage of the newest versions of Nexus3, prior to upgrading to version 3.71 or later, migrating the Nexus database from **OrientDB** to **H2** is required.

The *ge-nexus* chart version 1.0.12 (also part of AMP version 3.2.1) provides support to migrating data by including the Nexus Migrator tool from Sonatype into the GE-maintained docker image for Nexus 3.70.3. The steps presented below are based on the Sonatype documentation pages about Data backup and Migrating data with Nexus3 version 3.70.

> The upgrade to *ge-nexus* version 2.0.0 (or AMP to version 4.0.0) can only be done if the migration of OrientDB to H2 was done previously. The OrientDB-to-H2 migration steps can only be applied with *ge-nexus* chart version 1.0.12 (or AMP version 3.2.1).

11.3.1. Migration Steps

Step 1

Create a Complete Backup of DB in the Nexus UI

- Log in to Nexus with a privileged or admin user.
- Create a backup task in Nexus Task: Admin-export databases for backup
- Ensure all necessary components and configurations are included

Step 2

Nexus Pod Connectivity

Before proceeding with the migration steps, you must connect to the Nexus pod on the Kubernetes cluster. Use the following commands to gather information about the pods in your specified namespace and to connect to them.

```
# Command to get the list of pods in the specified namespace
kubectl get pods -n <amp-namespace>

# Command to get details of a specific pod
kubectl describe pod <pod-name> -n <namespace>

# Command to connect to a specific pod's container
kubectl exec -it -n <amp-namespace> <nexus-pod-name> -c <nexus-container-name> -- sh -c "clear; (bash
|| ash || sh)"
```

Step 3

Prepare Backup for Migration

- Copy the entire backup to a clean, separate location on the filesystem.
- This precaution ensures any extraction does not affect the existing production system.
- Copy backup to a safe location, example:

```
cp -r /nexus-data/<path-to-backup-data> /tmp/backup
```

Step 4

Shut Down the Nexus Repository

- To safely begin the migration process, stop the Nexus service by running the following command:

```
/opt/sonatype/nexus/bin/nexus stop
```

Step 5

Run the Migration Tool

- Along with the amp-nexus image version 3.70.3, the `nexus-do-migrator.jar` is available at:

```
/opt/sonatype/utilities/nexus-do-migrator-<version>.jar
```

- Navigate to the directory where the backup is stored and run the migration tool with the following command:

```
java -Xms160m -Xmx180m -XX:+UseG1GC -XX:MaxDirectMemorySize=256m \
-jar /opt/sonatype/utilities/nexus-db-migrator-3.70.3-01.jar \
--migration_type=h2
```

- Adjust memory settings as needed to fit the server's available resources.
- Replace `<version>` with the actual version number of the migrator jar.

Step 6

Copy the Resulting Database File

- The default path for `$data-dir` is `/nexus-data`.
- After migration, move the generated `nexus.mv.db` file to the `$data-dir/db` directory:

```
cp nexus.mv.db $data-dir/db/
```

Step 7

Update Nexus Configuration

- Edit the `$data-dir/etc/nexus.properties` file to enable the new datastore by adding the following line:

```
nexus.datastore.enabled=true
```

Step 8

Start the Nexus Repository

- Once configuration changes are complete, start the Nexus service:

```
/opt/sonatype/nexus/bin/nexus start
```

Step 9

Verify Migration Success

Before proceeding with the upgrade to Nexus versions (and AMP versions) based on Nexus 3.71 or later, it is essential to verify that all data has been successfully migrated.

- Log in to the Nexus UI and confirm the following elements:
  - All repositories are present and accessible.
  - All components and artifacts are intact and correctly associated with their respective repositories.
  - User permissions and access controls are correctly configured.
  - Any scheduled tasks or configurations from the previous setup are functional.

Once the verification is complete and all data is confirmed, your Nexus repository will be ready for the upgrade to **Java 17**.

11.4. AMP 3.1.2 (and *ge-jenkins* 5.4.2): Jenkins controller and PinK agent upgraded to use Java 17

Starting with *ge-jenkins* 5.4.2 (part of AMP 3.1.2), the Jenkins components use Java 17 as JRE. This will affect Jenkins pipelines for which an older version of Java is required. Refer to Jenkins documentation on how to configure a JDK in Jenkins.

11.5. AMP 3.0.0 (and *ge-gitea* 10.1.3): Gitea 10.x Helm Values

Gitea's Helm chart values introduced breaking changes with their 10.x version. See https://gitea.com/gitea/helm-chart#upgrading and click on the 10.0.0 version.

12. Security Considerations

There are two main components to consider when securing your deployment environment:

- AMP itself
- The Installation Execution machine, from where AMP was initially deployed

12.1. AMP

AMP is deployed to a Kubernetes Cluster, and uses Kubernetes features to protect your deployment system:

- Access to AMP components is ONLY via https through a single Ingress Controller
- Secrets (including encryption keys and certificates) are managed by Kubernetes

12.1.1. Open Ports

All traffic to AMP is routed through an encrypted Ingress port. By default, this is port 443.

For initial installation you must be able to supply the cluster with images to run. This either requires a 'registry' to be available where images can be downloaded from, or direct access to at least one node in the cluster. This can be achieved using ssh (defaults to port 22). Note that this access is only required temporarily - once AMP is running the port can be closed. AMP itself will then act as a registry from which images can be downloaded.

12.2. Installation Bootstrap\Execution machine

The initial deployment of AMP should be run from a machine with connectivity to the targeted environment. This machine is referred to as the Installation Bootstrap\Execution machine.

Especially in the case of airgap installation, this machine should be treated with the upmost caution. On its file system, it is expected that there will be critical, sensitive information including encryption keys and credentials. Ensure that only a restricted set of trusted users have access to the machine.

12.2.1. Open Ports

It is assumed that the Target node would be accessed via ssh, on the default port (22).

13. Secret Management

It is important that all secrets/credentials used within AMP are stored in a secure way. The current method for doing this is to store secrets in Jenkins. This will allow those secrets to be used within Jenkins while at the same time masking them in any log output.

13.1. Types of Credentials

Jenkins supports multiple types of credentials. The table below details the main ones:

Type	Description
Username and Password	Used to store a username and password together.
Secret Text	Typically used to store an API token such as Gitea access token
Secret File	Used to store the contents of a file. For example, storing a Kubeconfig file.

13.2. Secret Masking

When secrets are used in a Jenkins pipeline, Jenkins will automatically mask the value of the secret in the log output. Where a secret value would appear in the log output, Jenkins will replace the value with `*****`.

13.3. How to Add a New Secret

1. Log into Jenkins
2. Click `Manage Jenkins` → `Manage Credentials`
3. Click on `Jenkins` under `Stores Scoped to Jenkins` then click on `Global credentials`
4. Click `Add Credentials` to the left of the screen
5. Under `kind` select the type of secret
6. Fill in details depending on selected kind.
7. Ensure that the given `ID` is appropriate to identify the secret as this is what is used to retrieve the secret. This ID can not be changed after creating the secret. If no ID is given, then Jenkins will generate a unique ID.
8. Click OK

13.4. Using a Stored Secret in a Jenkins Job

The typical approach to using a secret within a Jenkins job is to first store it in an environment variable and then use that environment variable when needed. For example, if you have a credential called `GITEA_CREDENTIALS` then the below example would be in your Jenkinsfile.

```
pipeline{
    environment {
        GITEA_CREDS = credentials('GITEA_CREDENTIALS')
    }
    stages{
        stage('Example')
            steps{
                sh "echo ${env.GITEA_CREDS_USR}"
                sh "echo ${env.GITEA_CREDS_PSW}"
            }
    }
}
```

To access the username or password of the username/password secret, `_USR` or `_PSW` is added at the end. As expected the above example would print:

`****`

`****`

14. Troubleshooting

14.1. Certificates Are Not Trusted By Browser

By default, AMP is installed with a self-signed certificate issuer (through the use of the cert-manager), which will not be trusted by your browser. User machines should be configured to trust this certificate.

Alternatively, as presented under the 'Configuration' section, under 'Certificates manager', other types of certificates issuers may be used:

- *ACME issuer* assumes requesting new certificates from the internet as further detailed below;
- *CA Issuer* allows you to supply the relevant certificate and key (ssl.key.pem and ssl.cert.pem) to the Kubernetes cluster and have it bound to the AMP installation, would it by a certificate from your preferred Certificate Authority (that your browser already trusts) or from another CA.

When using the ACME issuer, the configuration for AMP will request new certificates from the internet (e.g. Let's Encrypt).

If you can access the internet but the certificates are still not trusted, you can check on the progress of your 'order' for new certificates from Let's Encrypt. The process whereby certificates are ordered is as follows:

1. CertManager sends an 'order' to Let's Encrypt for a new certificate
2. CertManager also creates a service which will respond to 'Challenge' requests
3. Let's Encrypt checks you really own the domain name the request was for. It does this by issuing a 'Challenge' request
4. Assuming the challenge is passed Let's Encrypt will issue a new certificate
5. CertManager will store the certificate (in a secret) ready to be used

Assuming you have installed AMP into a namespace called 'amp', you can check on the current status of the certificates, orders and challenges with:

```
kubectl get certificates -n amp
kubectl get orders -n amp
kubectl get challenges -n amp
```

If the orders and challenges exist but are not being fulfilled, then remove the problem order and challenge and remove the certmanager pod to force it to restart.

Find the CertManager pod with:

```
kubectl get pods -n amp | grep cert
```

Then encourage CertManager to try again with:

```
kubectl delete order -n amp <problem-order>
kubectl delete challenge -n amp <problem-order>
kubectl delete pod -n amp <cert-manager-pod>
```

14.2. Jenkins - max_user_namespaces Error

If faced with the following error while attempting to use Podman/Docker:

```
user namespaces are not enabled in /proc/sys/user/max_user_namespaces
Error: cannot re-exec process
```

It means that max_user_namespaces is not enabled (the corresponding file having a value of 0). Therefore, all that needs to be done is to add a value (for example, 3000) onto the nodes of the Kubernetes cluster, for example:

```
echo "3000" > /proc/sys/user/max_user_namespaces
```

14.3. Jenkins - Timing Out While Pulling Docker Image

Depending on the system config and the size of the image being pulled, there might be some scenarios when Jenkins runs out of memory or cpu limit might be too low. For this, as supplied in the manifest, the memory limit is set to 1G. One can choose to make this even higher, if the cluster permits it. For reference, please check chart values.

```
  agent:
    resources:
      limits:
        memory: "1Gi"
```

14.4. AMP - Checking the Chart Version from Current Deployment

In order to find out the Chart version of the current AMP deployment, the following command can be used:

```
helm list -n <release-namespace>
```

It should result in a list with all helm releases into the mentioned namespace and providing in the *CHART* column the chart versions used for these:

```
helm list -n amp
NAME    NAMESPACE       REVISION        UPDATED                                   STATUS          CHART
APP VERSION
amp     amp             1               2025-01-14 21:07:11.669480635 +0200 EET deployed        AMP-
4.0.1       4.0.1
```

15. Licenses

15.1. Third Party Components

Component	License	URL
Jenkins Controller	MIT, Free	https://www.jenkins.io/license/
Jenkins Pink Agent	MIT, Free	https://github.com/jenkinsci/docker-inbound-agent/blob/master/LICENSE
Nexus Repository Manager	OSS Edition, Free	https://www.sonatype.com/products/repository-oss
Gitea	MIT, Free	https://github.com/go-gitea/gitea/blob/main/LICENSE
Cert-manager	Apache 2.0, Free	https://github.com/cert-manager/cert-manager/blob/master/LICENSE
Postgres Operator	MIT, Free	https://github.com/zalando/postgres-operator/blob/master/LICENSE
Argo CD	Apache 2.0, Free	https://github.com/argoproj/argo-cd/blob/master/LICENSE

15.2. Third Party Component Premium Licenses

Support and additional features can be purchased for the Nexus if required, with a 'Pro' license

Component	Details
Nexus Repository Manager	https://www.sonatype.com/products/pricing

16. Migration from PDI to Helm AMP

16.1. Overview

With version 3.0.0 (and later), the Automation Management Platform (AMP) installation has changed from AMP 2.x (and earlier) as it now relies on Helm charts (and no more on PDIs). Migrating a PDI-based AMP to the Helm-based one is a requirement in order to benefit from the latest versions. The information in this page are minimal steps identified for this migration with no data loss.

To achieve this, the following steps shall be observed:

1. Ensure the system can download the required Images
2. Retrieve the latest AMP helm chart
3. Prepare back-ups (where required) and installation configurations
4. Migrate the existing installation with the helm chart
5. Restore back-ups where required

16.2. Image Availability

The functional components of AMP run in containers, downloaded as Images. If your infrastructure has access to GE Digital's Grid Artifactory (https://dig-grid-artifactory.apps.ge.com/) then no work is required - the new Images required for the upgrade will be automatically available.

Where downloading directly from Grid Artifactory is not possible then the 'Airgap Process' must be used to install images into your local registry.

16.3. Prerequisites

The migration process requires the following tools to be installed on the your Execution/Bootstrap machine:

- Helm
- kubectl

16.4. Prepare AMP Installation Migration

Before migrating the helm-based AMP, we must make sure to

- back-up configurations of the AMP components instances
- prepare the *overrides-values.yaml* to be used for the AMP installation

As a generic recommendation, make a backup of the data bound to each of the AMP components. Still, following the indications for applying the migration, only Jenkins would require backing up configurations especially about plugins installed since the initial Jenkins instantiation. As a solution presented under https://www.jenkins.io/doc/book/system-administration/backing-up/, the plugin https://plugins.jenkins.io/thinBackup/ can be used:

- Install the 'thinBackup' plugin
- Go to 'Manage Jenkins > Tools and Actions > ThinBackup'
- Click 'Settings':
  - Field 'Backup directory' to '${JENKINS_HOME}/back-ups'
  - Field 'Max number of backup sets' to a fix number (different from default -1)
  - Option 'Wait until Jenkins is idle to perform a backup' unchecked
  - Option 'Backup plugins archives' checked
  - Other backup may be checked if considered necessary !!!
  - Click 'Save'
- Click 'Backup now'

Then, the *overrides-values.yaml* shall be prepare. The example below is the minimal version to be used. Make to sure to adapt it to match your needs.

```yaml
global:
  jenkinsHostName: "jenkins.amp.myco.com"
  giteaHostName: "gitea.amp.myco.com"
  nexusHost: "nexus.amp.myco.com"
  nexusUiHost: "ui.nexus.myco.com"
  nexusDockerPushHost: "nexus.push.amp.myco.com"
  nexusDockerPullHost: "nexus.pull.amp.myco.com"
  ingress:
    className: &ingressClassName "nginx"
  certificate:
    issuer:
      name: &ampCertIssuer "amp-issuer"
  storageClass: &ampStorageClass "local-path"

ge-jenkins:
  jenkins:
    persistence:
      enabled: true
      create: false
      existingClaim: "amp-jenkins-pvc"
      storageClassName: *ampStorageClass
    controller:
      admin:
        password: "my-jenkins-P@ssw0rd"
      ingress:
        ingressClassName: *ingressClassName
        annotations:
          cert-manager.io/issuer: *ampCertIssuer

ge-nexus:
  repositories: []
  initAdminPassword:
    enabled: false
  persistence:
    enabled: true
    existingClaim: amp-nexus-sonatype-nexus-data-amp-nexus-sonatype-nexus-0
    storageClass: *ampStorageClass

ge-gitea:
  gitea:
    gitea:
      admin:
        username: 'root'
        password: "my-gitea-P@ssw0rd"
    persistence:
      claimName: "gitea-shared-storage"
      storageClass: *ampStorageClass
    ingress:
      className: *ingressClassName
      annotations:
        cert-manager.io/issuer: *ampCertIssuer
```

- The example above uses YAML anchor to avoid setting the same piece of information multiple times.
- For the Jenkins and Gitea admin. passwords need to be provided
  - To avoid hard-coding them into the the YAML file use the '--set' flags to set these variables with the helm installation command mentioned later on this page.
- Make sure that the PVC names for Jenkins, Gitea and Nexus to be re-claimed are the ones from your current installation

We recommend using a Source Control system (Git) for maintaining your *values.yaml* files, which assists with 'three-way merge' of changing files. The goal is to ensure that the changes made to the original file are re-applied for the future upgrades.

Known limitation

The current *overrides-values.yaml* assumes that you are using AMP 3.0.1 Helm chart (or later)

As-is, when using AMP 3.0.0 Helm chart, the migration will only work if `ge-nexus.statefulset.enabled=false` is added to the values but it will have for effect to change the *StatefulSet* original deployment into a *ReplicaSet* one for the Nexus component. This is due to a limitation discovered in the Nexus Helm chart (ge-nexus 1.0.2), which has been remedied from AMP 3.0.1 Helm chart (thus, in ge-nexus 1.0.4 and later).

16.5. Apply The AMP migration

The migration in itself takes four steps:

- Uninstalling the AMP components using helm
- Updating the Cert. Manager Custom Resource Defintions (CRDs)
- Changing attributes of Persistent Volume Claims (PVCs)
- Installing AMP with the new helm chart

The steps describe below assumes:

- the current PDI-based AMP installation to be under the 'amp' namespace.
- The new AMP installation to have 'amp' for release name.
- Use of the '--kubeconfig' flag with the kube configuration file for the targeted cluster (if necessary).

16.5.1. Step #1 - Uninstalling PDI-based AMP components

Execute the following command against the cluster your AMP installation resides on:

```
helm uninstall amp-nexus amp-jenkins amp-gitea amp-zalando amp-cert-manager -n amp
```

16.5.2. Step #2 - Updating Cert. Manager CRDs

Execute the following command against the cluster your AMP installation resides on:

```
kubectl apply -f https://github.com/cert-manager/cert-manager/releases/download/v1.14.4/cert-manager.crds.yaml
```

> At the time of the creation of this document, the latest version was v1.14.4. Please use the version corresponding to the one for the 'ge-cert-manager' component of the AMP helm chart.

16.5.3. Step #3 - Changing PVCs attributes

Based on the PVC names set into the *overrides-values.yaml* prepared previously, attributes related to the release name will have to be aligned with the 'amp' release name for them on the cluster.

For the Gitea storage PVC (named 'gitea-shared-storage' in the example above), the following attributes shall be edited from 'amp-gitea' to 'amp':

```
metadata:
  annotations:
    meta.helm.sh/release-name: amp
```

Similarly to the Gitea PVC, the Jenkins one (named 'amp-nexus-sonatype-nexus-data-amp-nexus-sonatype-nexus-0' in the example above), the following attributes shall be edited from 'amp-jenkins' to 'amp':

```
metadata:
  labels:
    release: amp
```

16.5.4. Step #4 - Installing Helm-based AMP

Using the *overrides-values.yaml* prepared earlier on, the following command shall be applied:

```
helm upgrade amp ./AMP-3.0.1.tgz -n amp --install -f overrides-values.yaml --timeout 15m --wait --wait-for-jobs --set 'ge-cert-manager.cert-manager.installCRDs=false'
```

> - The flag `--set 'ge-cert-manager.cert-manager.installCRDs=false'` is required to avoid CRDs installation conflict (since already existing on the cluster) as this variable is defaulted to `true` from AMP 3.0.1
> - At the time of writing this document, the AMP latest version was 3.0.1.

16.6. Restore back-ups (where required)

As mentioned in the preparation section of the present document, a back up is necessary for Jenkins

(especially for the Jenkins plugins). The restoration of this backup will be done with the 'thinBackup' plugin as follow:

- Install the 'thinBackup' plugin
- Go to 'Manage Jenkins > Tools and Actions > ThinBackup'
- Click 'Restore':
  - Select the latest backup in 'restore backup from'
  - Option 'Restore next build number file (if found in backup)' checked
  - Option 'Restore plugins' checked
  - Click 'Restore'
- Go to 'Manage Jenkins > Plugins'
  - Verify that the 'Download progress' have finalized
    - Check the 'Restart Jenkins...' option if required by plugins